SAP University Competence Center
Chair for Information Systems and Business Process Management
TUM School of Computation, Information, and Technology
Technical University of Munich

Advanced Practical Course – Next Generation Data Center Operations for SAP Enterprise Software (IN2128, IN2106, IN212810)

Containers

Simon Fuchs, Maximilian Niedermeier, Noah Kim, Leon Kist

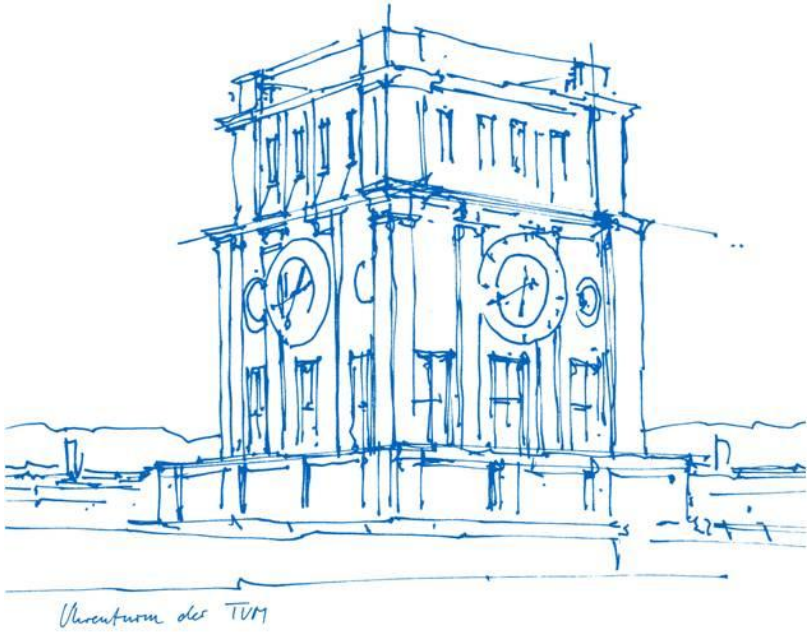
SAP University Competence Center

Information Systems and Business Process Management

TUM School of Computation, Information and Technology

Technical University of Munich

17.04.2026



Container Orchestration

Introduction

Outline

- Containers in General
  - Definition
  - Container vs VM
- Container Environment
  - Container Registries
  - Building and Running Containers
  - Docker & Podman
- Container Orchestration → **How are containers scaled?**
  - Hyperscalers & Cloud
  - Kubernetes

Introduction

Group exercise:

- Form groups of 3-5 people → Your table
- As a group: Think about scaling (10 minutes)
  - Come up with 2 **everyday concepts** (outside of computing) that are scaled to a large/massive degree
  - Answer:
    - **What** is the concept?
    - **Why** does it need to be scaled?
    - **How** is it scaled, i.e., what mechanism is used? (Horizontal vs. Vertical)
- Present your results

Introduction

Group exercise:

- Example: Competitive Football (Soccer)
  - **What** is the concept?
    - Competitive Soccer
  - **Why** is it scaled?
    - Millions of people (in Germany) play → cannot hold one big tournament/game
  - **How** is it scaled, i.e., what mechanism is used?
    - Multi-tier league system (up to 13) with promotion/demotion, from nation-wide down to regionally locked
    - Horizontal scaling (many small- to mid-sized leagues instead of one large one)

Introduction

We have moved from **manual** container creation to **tools** for building, running, and monitoring

However:

- Tools (e.g., Docker & Podman) still require direct creation and management (by the user)
- Feasible for small to medium amount of containers
- **Does not scale!**

Introduction

We have moved from **manual** container creation to **tools** for building, running, and monitoring

However:

- Tools (e.g., Docker & Podman) still require direct creation and management (by the user)
- Feasible for small to medium amount of containers
- **Does not scale!**

Solution → container orchestration tools

Hyperscaler & The Cloud

Who needs that many containers? And Why?

Hyperscaler & The Cloud

Who needs that many containers? And Why?

Regular company or datacenter:

- Dedicated (on-premise) servers
- Even if large number of applications exist → managed directly by teams

Problem: requires know-how, hardware expensive, long setup time, dedicated personal needed, …

Hyperscaler & The Cloud

Who needs that many containers? And Why?

Regular company or datacenter:

- Dedicated (on-premise) servers
- Even if large number of applications exist → managed directly by teams

Problem: requires know-how, hardware expensive, long setup time, dedicated personal needed, …

Solution → the cloud

Hyperscaler & The Cloud

What is the **cloud**?

- Applications are hosted externally
- Accessed over the internet
- (Often) Large dedicated datacenters in strategic locations
- Typically charge regular fees or depending on usage

Hyperscaler & The Cloud

Benefits (for users):

- No know-how required (for hardware and hosting) & no hardware costs
- Fast/immediate setup
- Managed externally → no personnel required
- Accessible for small companies (even in emerging markets) or private users
- Scaling

Hyperscaler & The Cloud

Problems (for users):

- Dependent on internet access
- Security and trust issues
- Dependency on providers
- Centralized hosting → single point of failure

Hyperscaler & The Cloud

Typical Services:

- Infrastructure as a service (IaaS) → IT infrastructure (e.g., computing, storage, …)
- Platform as a service (PaaS) → Tools and services to deploy applications (e.g., OS, runtime env., …)
- Software as a Service (SaaS) → Fully functional applications (e.g., ERP, CRM, …)

Hyperscaler & The Cloud

Who runs the cloud?

Hyperscaler & The Cloud

Who runs the cloud?

IBM Cloud

Sources: see references

Hyperscaler & The Cloud

Enterprise Datacenter vs Hyperscaler (cloud provider)

Enterprise Datacenter:

- Defined scope:
  - Number of applications
  - Nature of applications

- Highly Customized
- Limited number of applications
- Industry specific compliance
- Company specific bandwidth requirements
- Company specific redundancy requirements

Hyperscaler:

- Undefined scope:
  - (Unknown) Number of applications
  - (Unknown) Nature of applications

- High degree of **flexibility** required
- Millions of concurrent applications
- Global compliance
- Extreme bandwidth requirements
- High redundancy

Hyperscaler & The Cloud

How can hyperscalers/cloud providers provide these constraints?

Hyperscaler & The Cloud

How can hyperscalers/cloud providers provide these constraints?

- Containers → flexibility, isolation, efficiency, …
- Container orchestration tools → scaling, redundancy, …

Hyperscaler & The Cloud

How can hyperscalers/cloud providers provide these constraints?

- Containers → flexibility, isolation, efficiency, …
- Container orchestration tools → scaling, redundancy, …

→ Kubernetes

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Agenda

Agenda:

- History of Kubernetes
- What is Kubernetes?
  - Design
  - Architecture
- Features

Kubernetes | History

Kubernetes:

- Successor to *Borg:*
  - Developed at Google
  - Unified container-management system
  - Both long-running services and batch jobs
  - Utilizes Linux container support → Google contributed code to the Linux kernel

Kubernetes | History

Kubernetes:

- Kubernetes = “helmsman” (ancient Greek)
- Pitched in 2013 by Craig McLuckie, Joe Beda and Brendan Burns (Google)
- Released in 2014 → Open source
- Now: **second largest** open-source project in the world (after Linux)
- Cloud Native Computing Foundation graduated Project

Kubernetes | History

Cloud Native Computing Foundation:

- Part of the Linux Foundation
- Provide: support, oversight, and direction to **cloud native projects**
- Large amount of industry partners
- Vast landscape

Kubernetes | History

Cloud Native Computing Foundation:

CLOUD NATIVE LANDSCAPE

Source: https://landscape.cncf.io/images/logo_header.svg

Kubernetes | Design

Kubernetes:

- “[..] system for automating **deployment**, **scaling**, and **management** of containerized applications”
- (Some) Features:
  - Automated rollouts and rollbacks
  - Service discovery and load balancing
  - Horizontal scaling, vertical scaling
  - Secret and configuration management

Kubernetes | Design

Kubernetes:

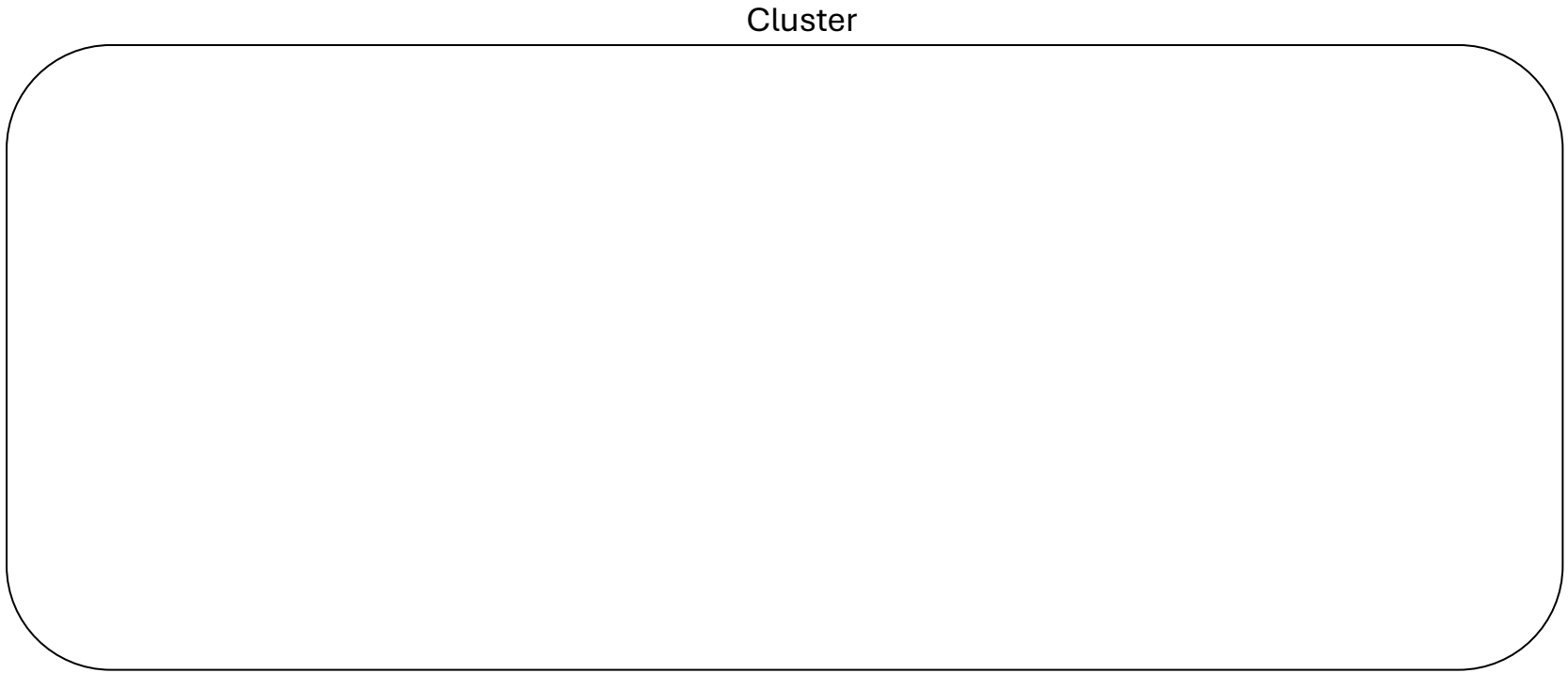
- Core **design principles**:
  - Immutability
  - Declarative configuration
  - Self-healing
  - Extensibility
  - Scalability

Kubernetes | Architecture

Kubernetes Cluster:

Kubernetes | Architecture

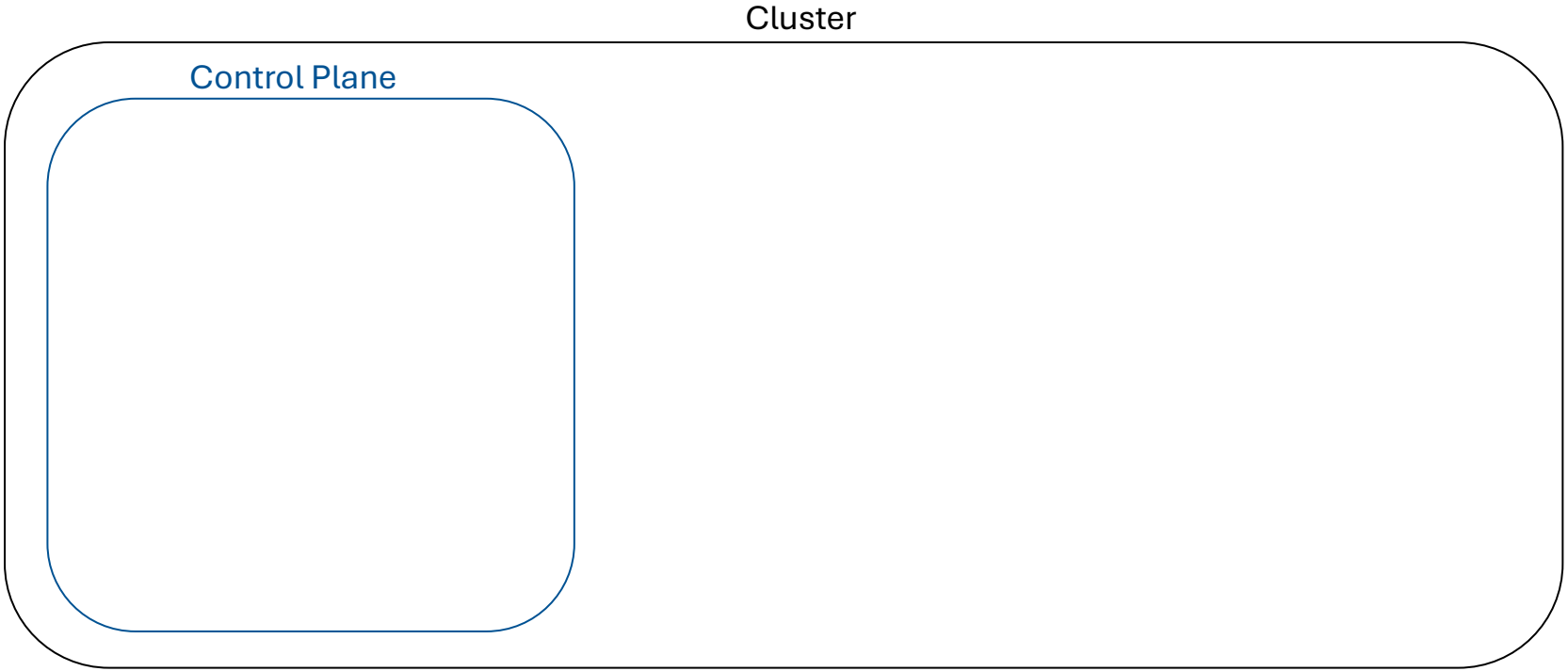
Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

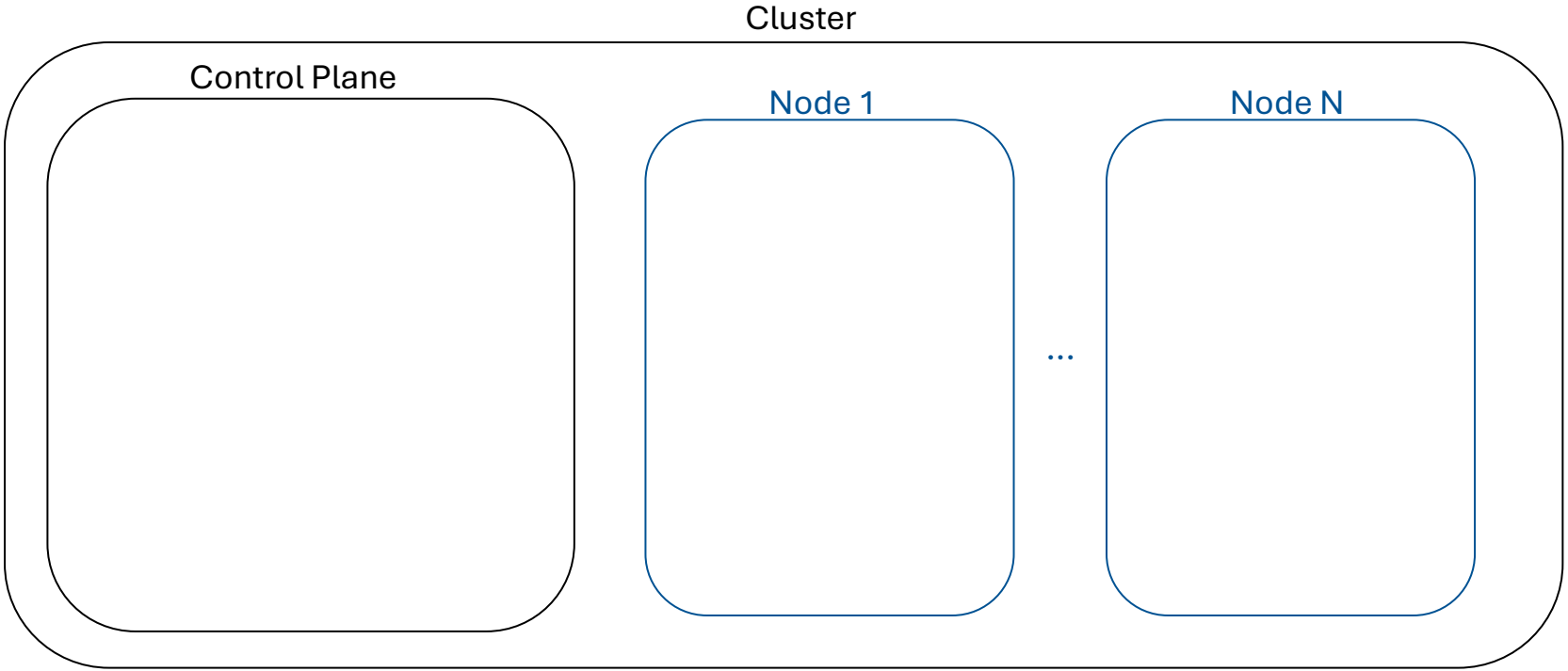
Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

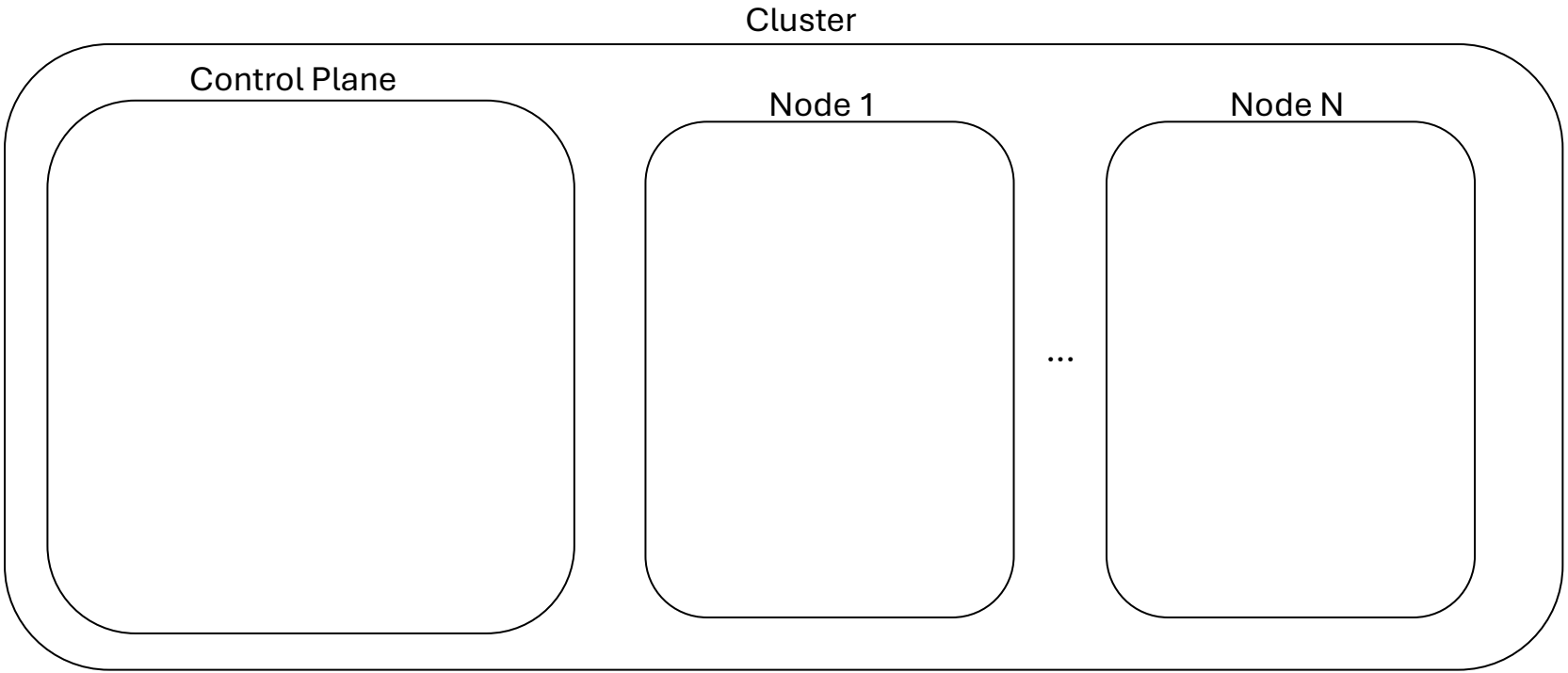
Kubernetes | Architecture

Kubernetes Cluster:

- **Nodes:**
  - Run containers/applications
- **Control Plane:**
  - Group of components that manages and orchestrates the cluster
  - Can run on any of the nodes in the cluster

Kubernetes | Architecture

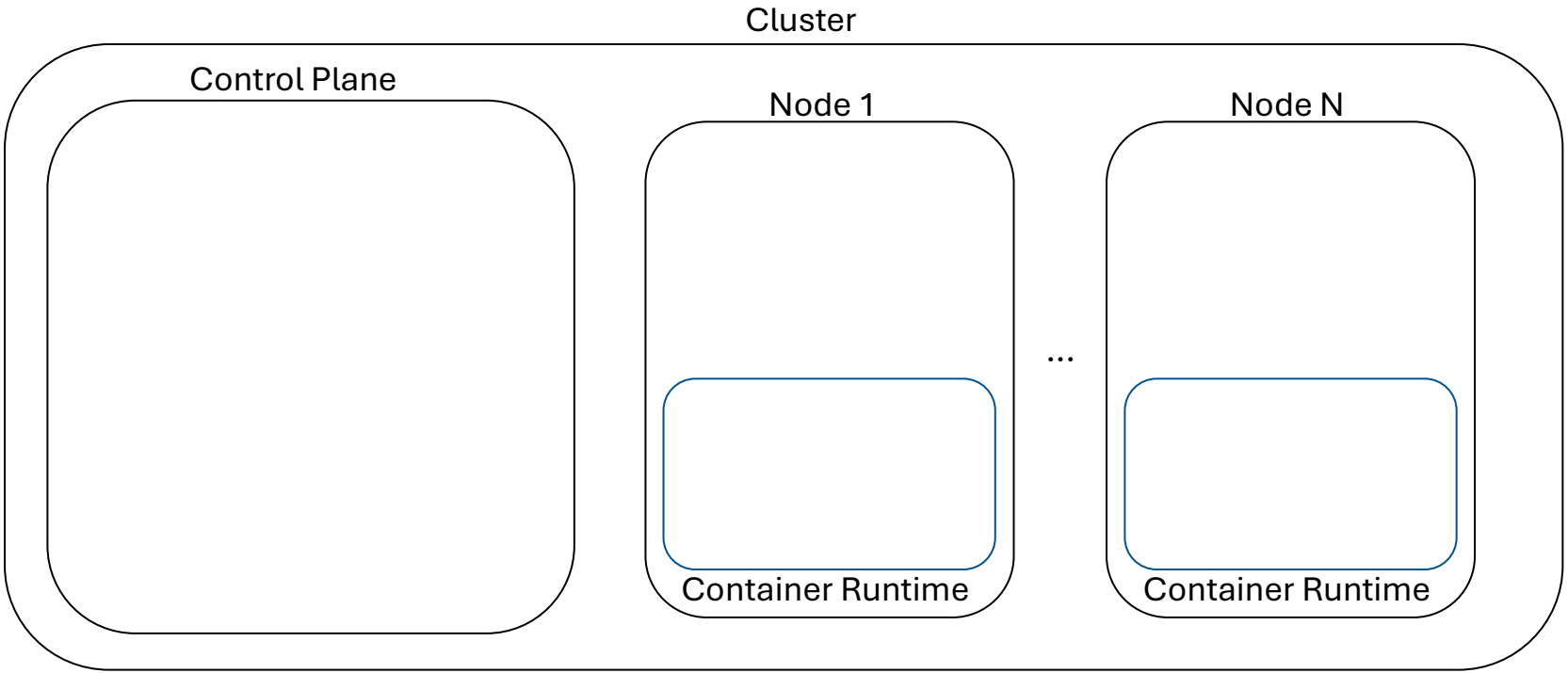
Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

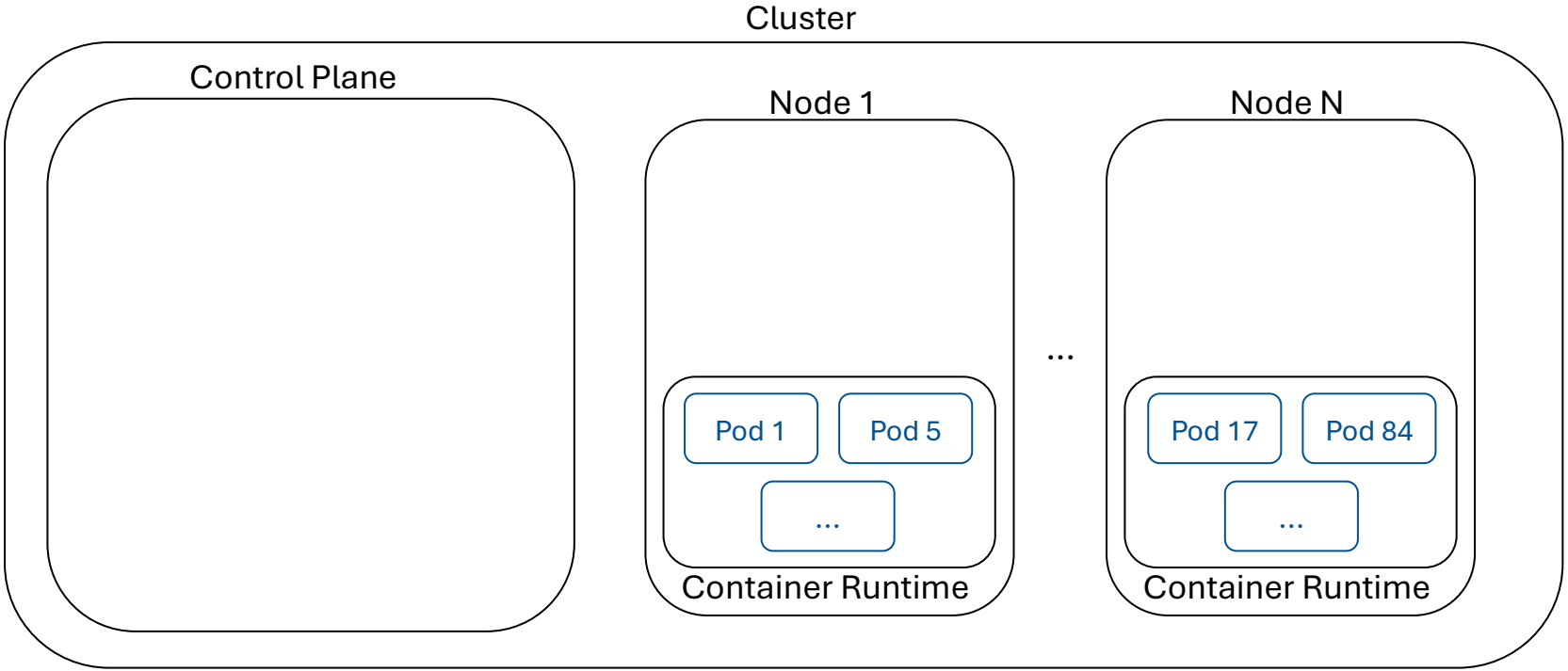
Kubernetes | Architecture

Container Runtime:

- Reminder → Responsible for:
  - Executing containers
  - Managing container lifecycle
- Has to implement Kubernetes Container Runtime Interface (CRI)
- Examples: containerd, CRI-O, Docker Engine

Kubernetes | Architecture

Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

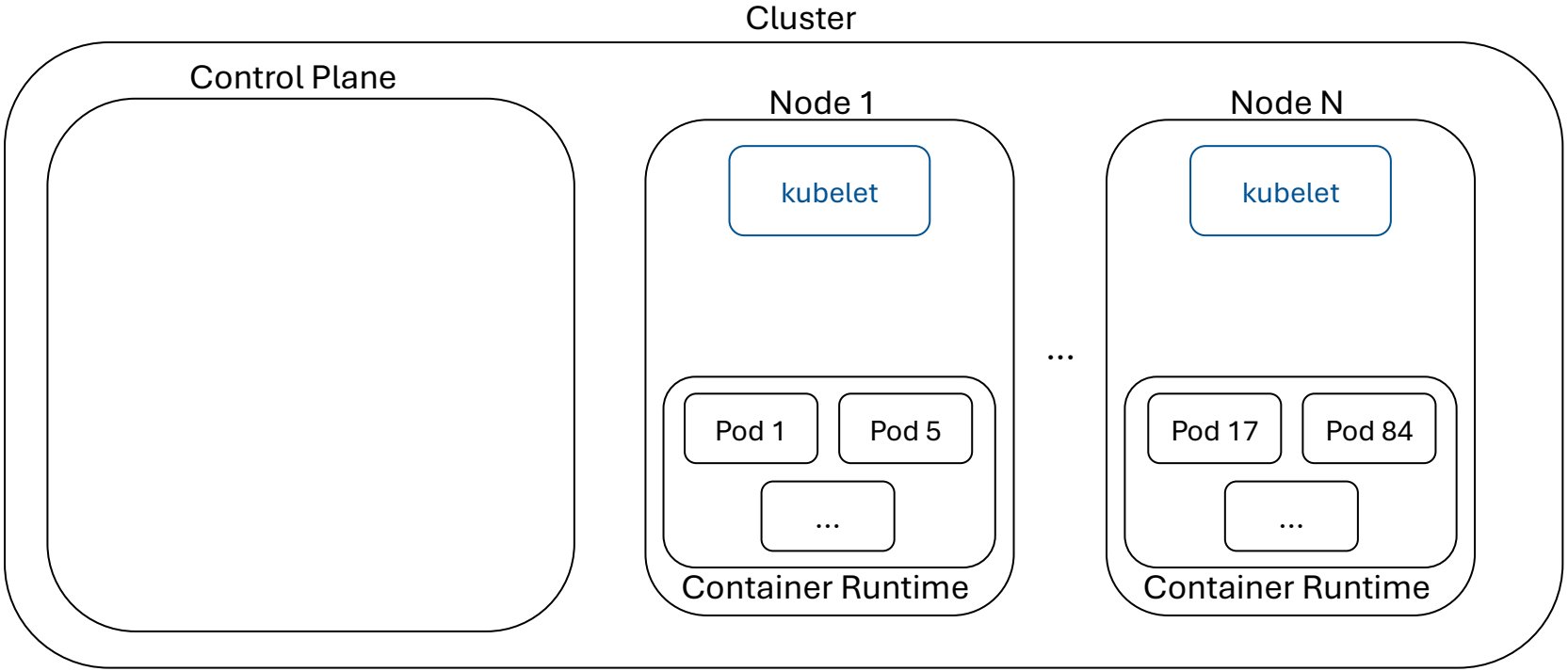
Kubernetes | Architecture

Pods:

- Smallest deployable unit of computation in Kubernetes
- Wrap a set of containers with shared namespaces and system volumes → always on the same node
- Usage:
  - Single-container pods: very common use case
  - Multi-container pods: containers that benefit from being on the same node or sharing resources

Kubernetes | Architecture

Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

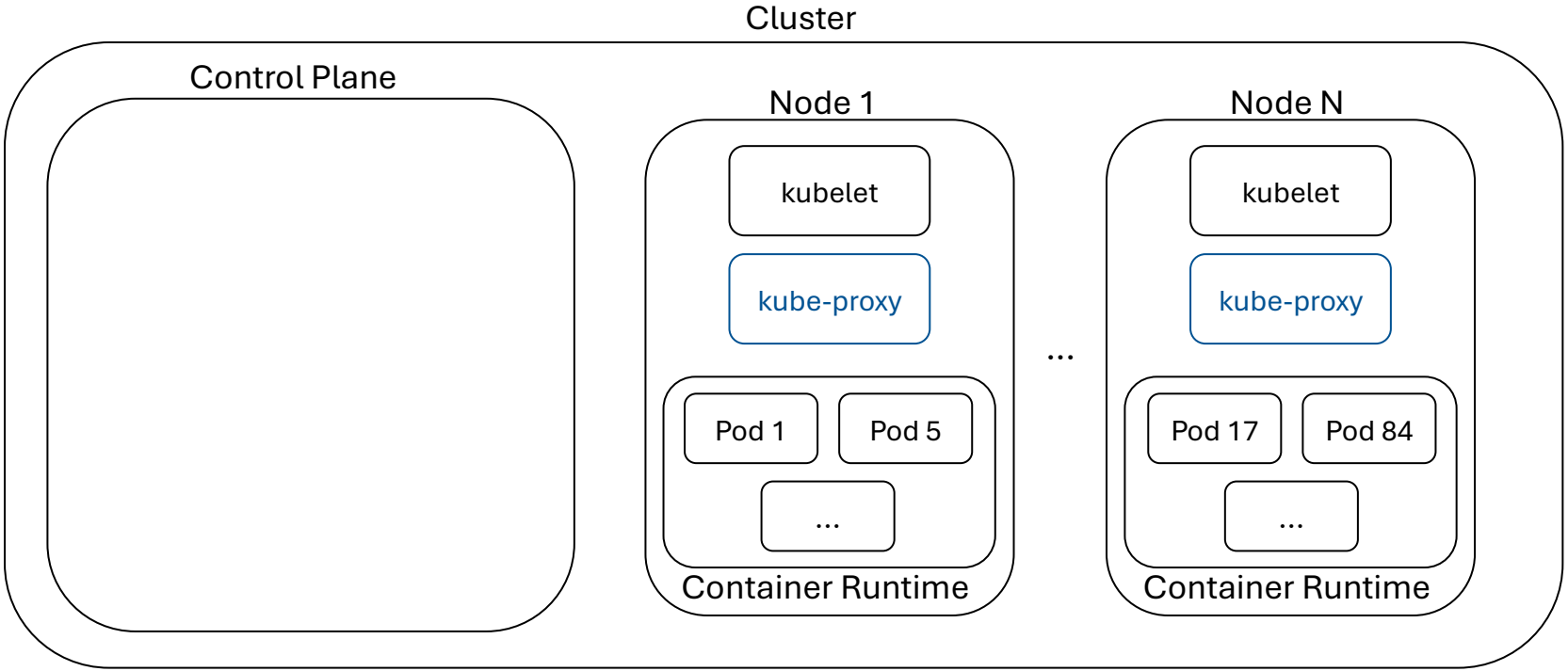
Kubernetes | Architecture

kubelet :

- Runs on each node
- Ensures that a specified set of containers are running and healthy
- Communicates with the control plane:
  - Registers the node with the cluster
  - Receives information about which containers to run (from the control plane)

Kubernetes | Architecture

Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

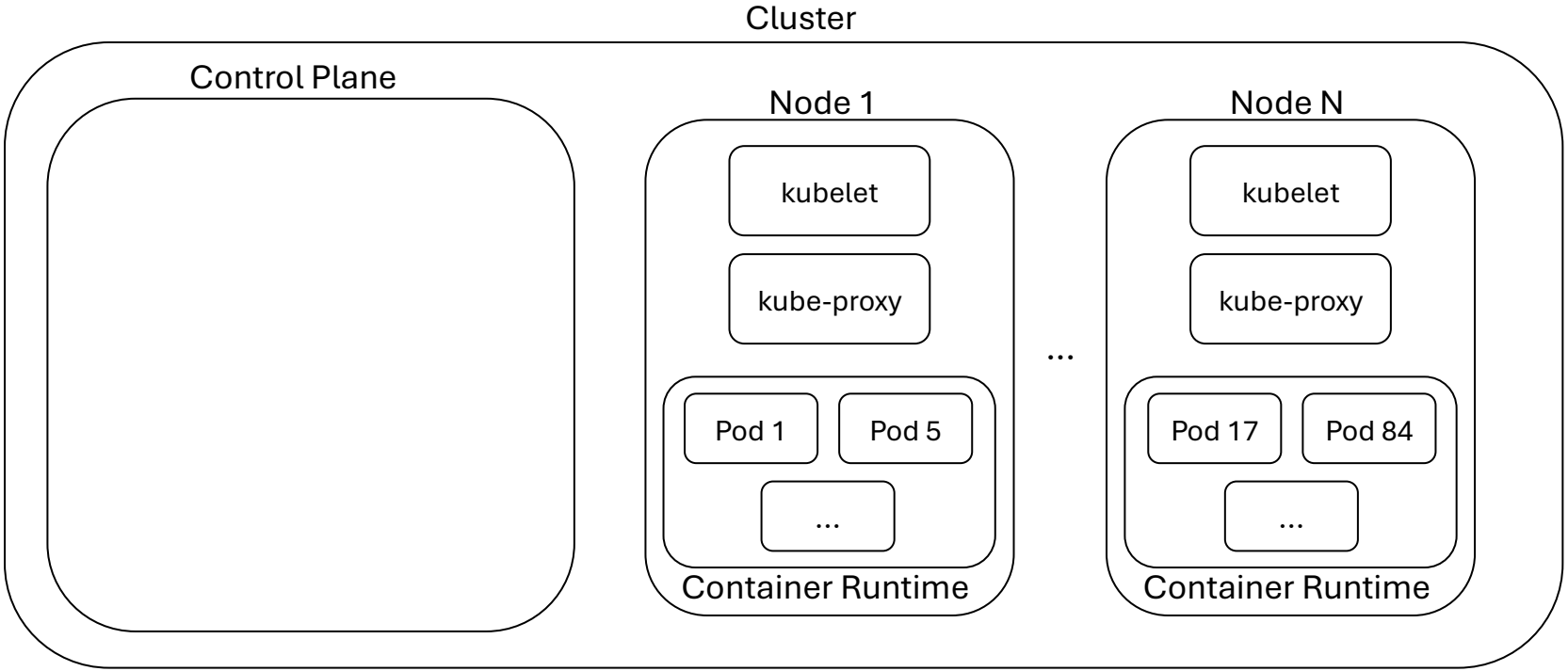
Kubernetes | Architecture

kube-proxy :

- (optionally) Runs on each node
- Network proxy
- Allows (network) communication with Pods from inside and outside the cluster
- Implements part of the functionality of Kubernetes Services & Service Discovery

Kubernetes | Architecture

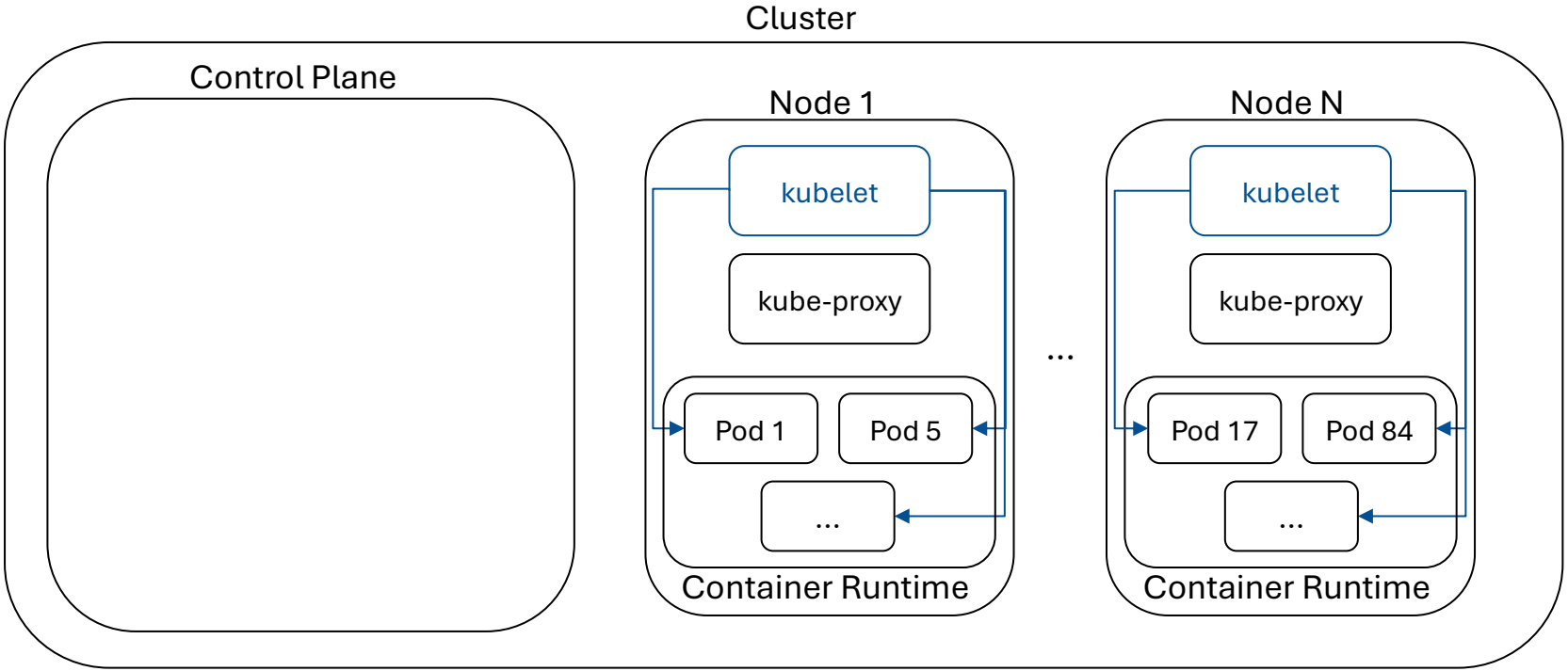
Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

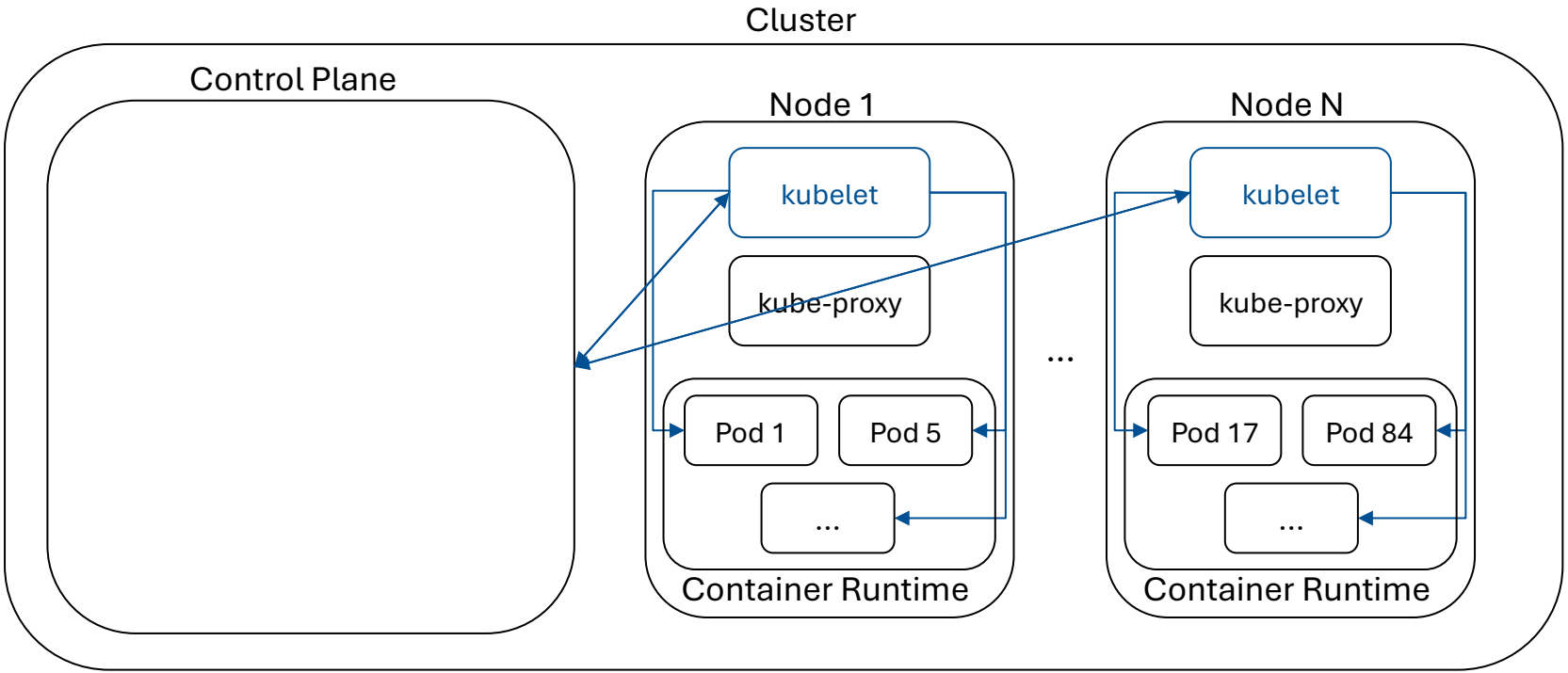
Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

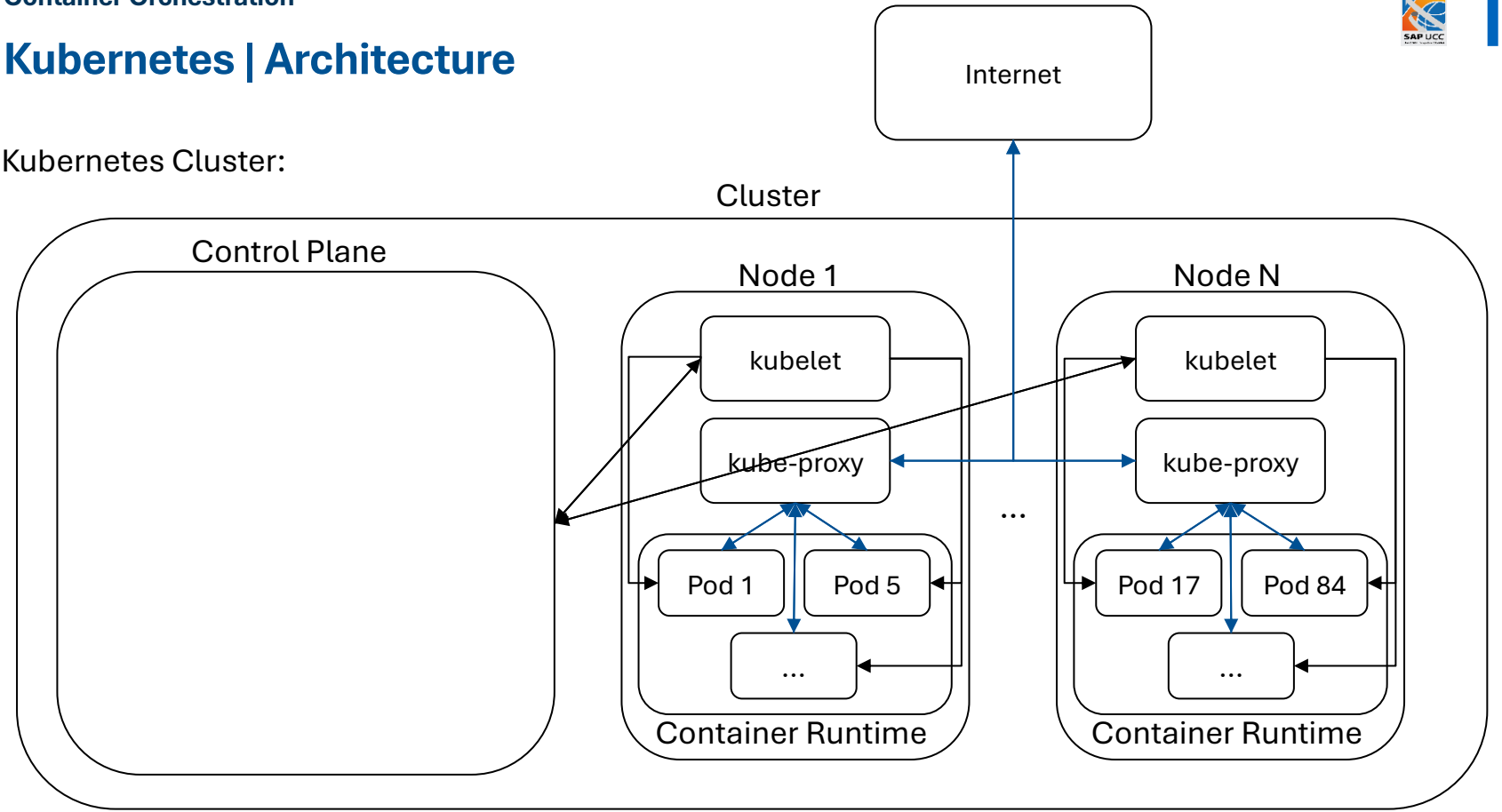
Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

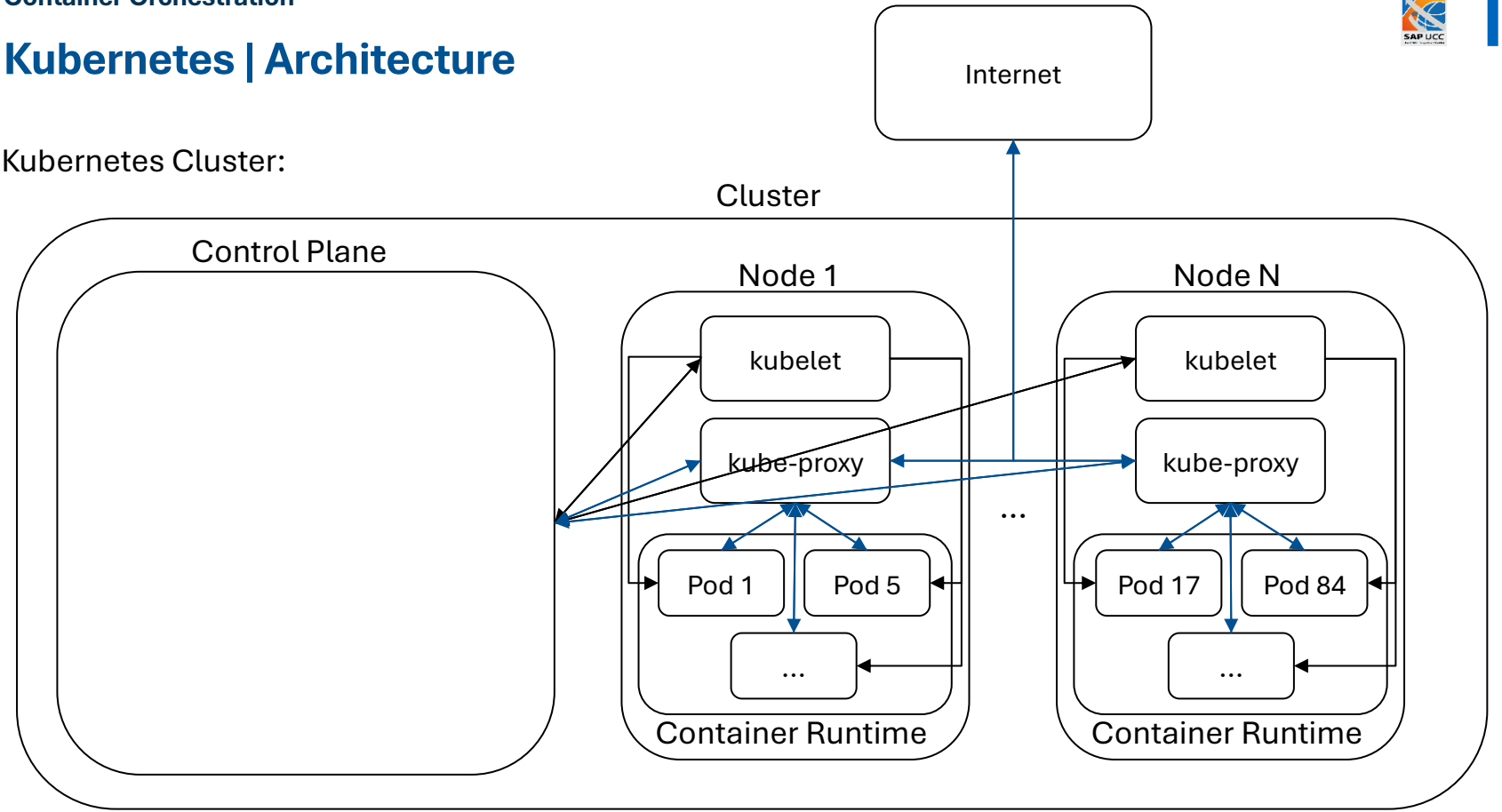
Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

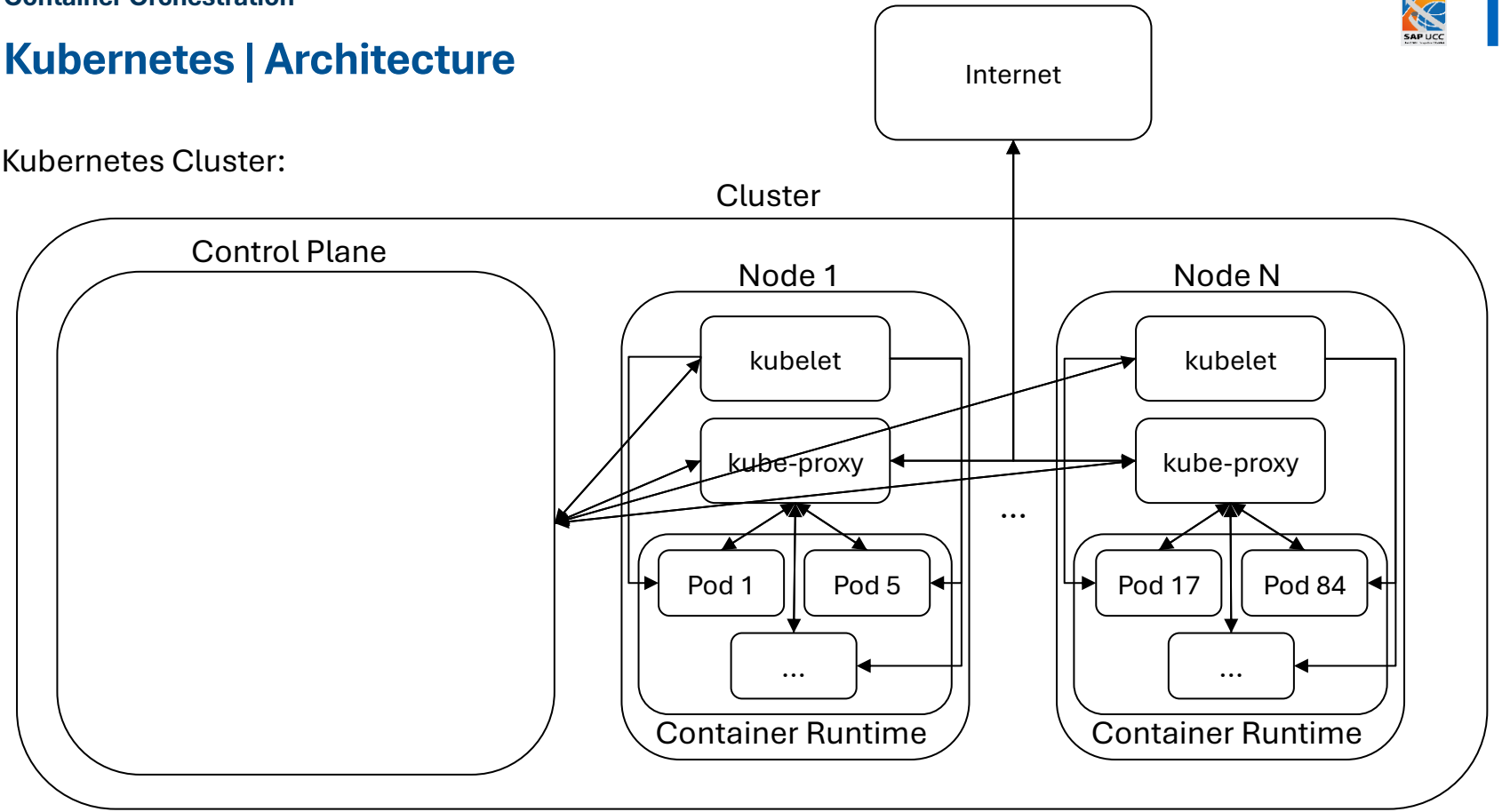
Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

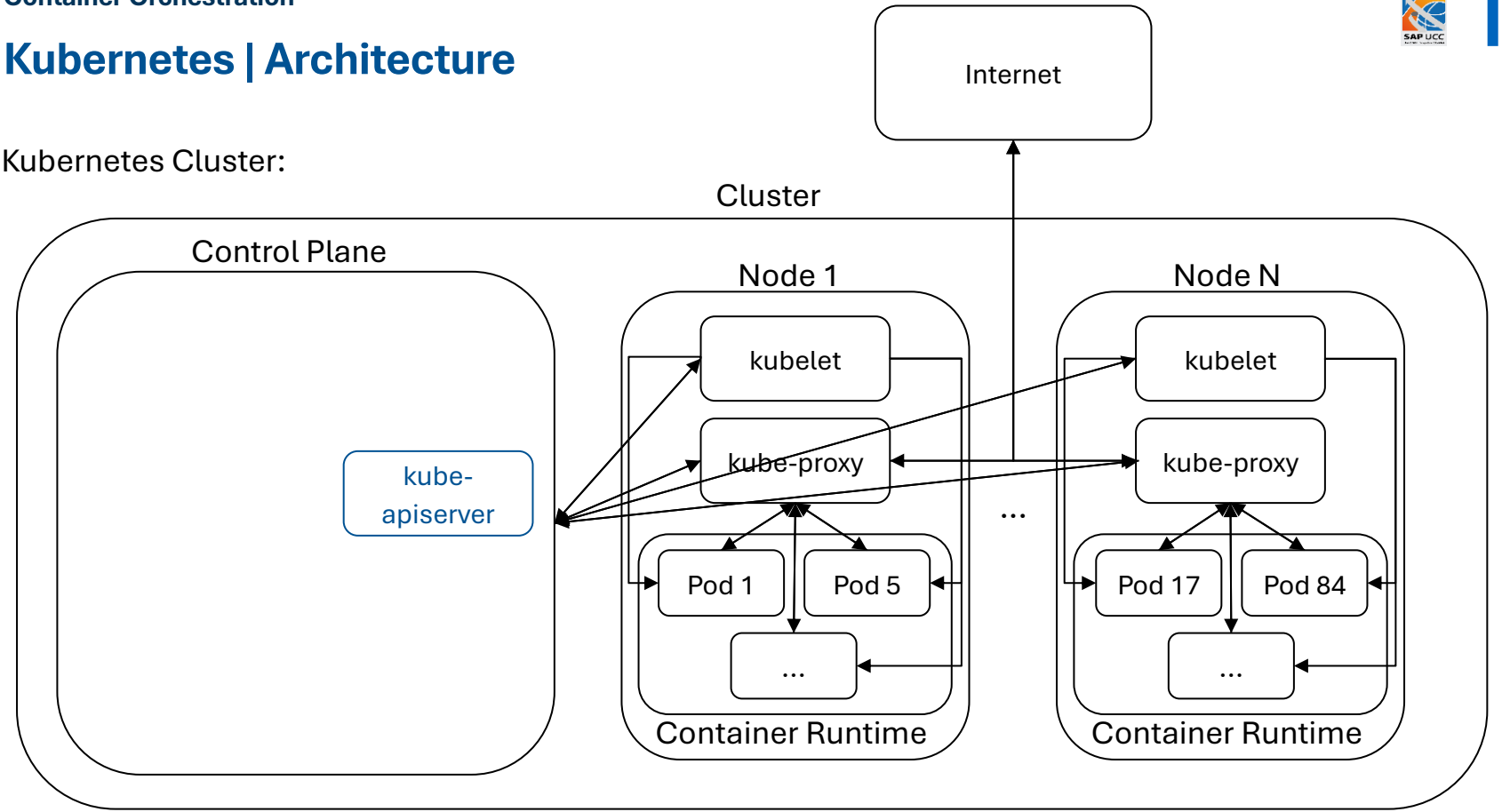
Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

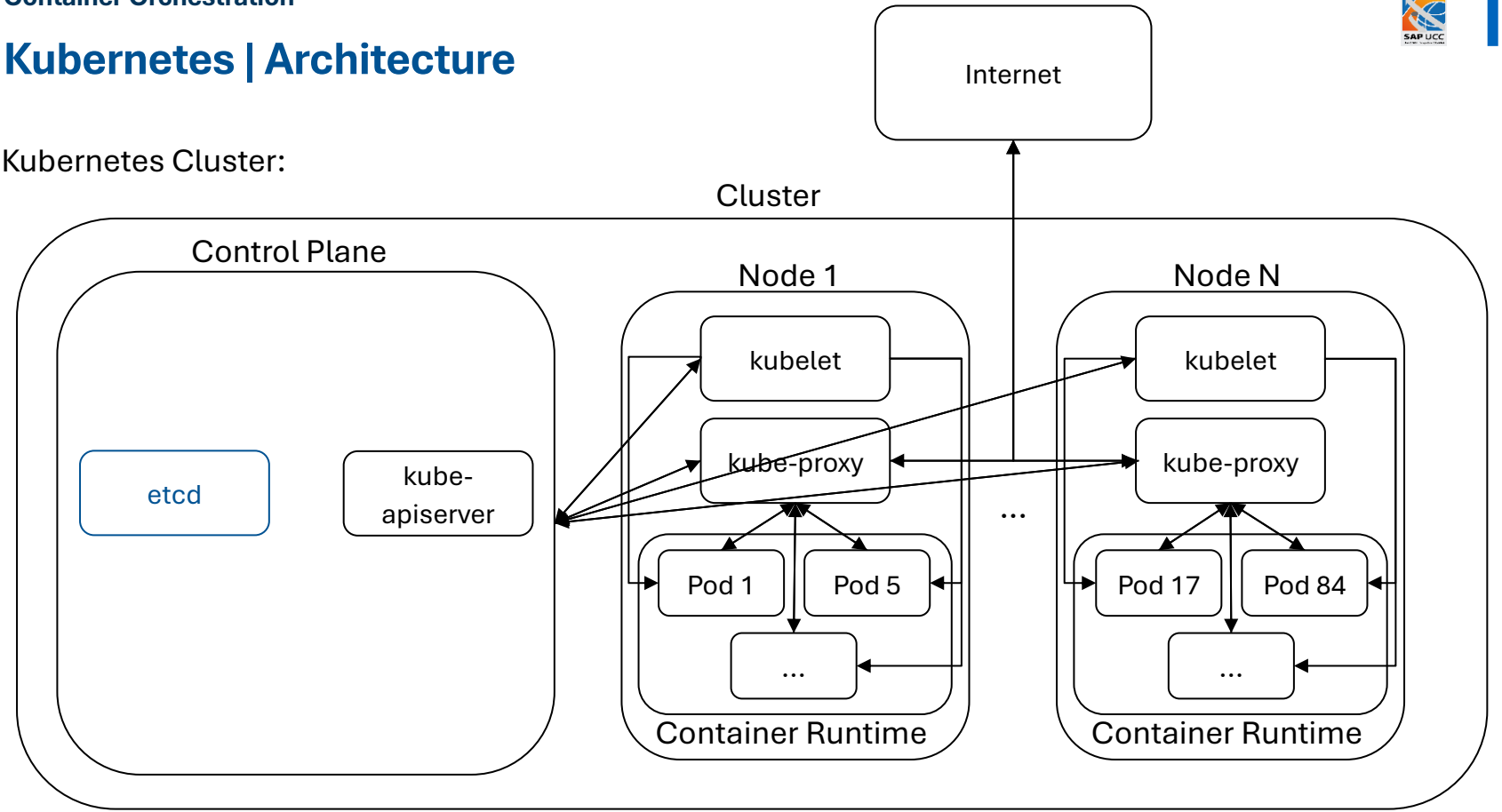
Kubernetes | Architecture

kube-apiserver:

- Exposes **Kubernetes API (REST)** to other components and users
- Frontend for control plane
- More instances can be deployed → horizontal scaling

Kubernetes | Architecture

Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

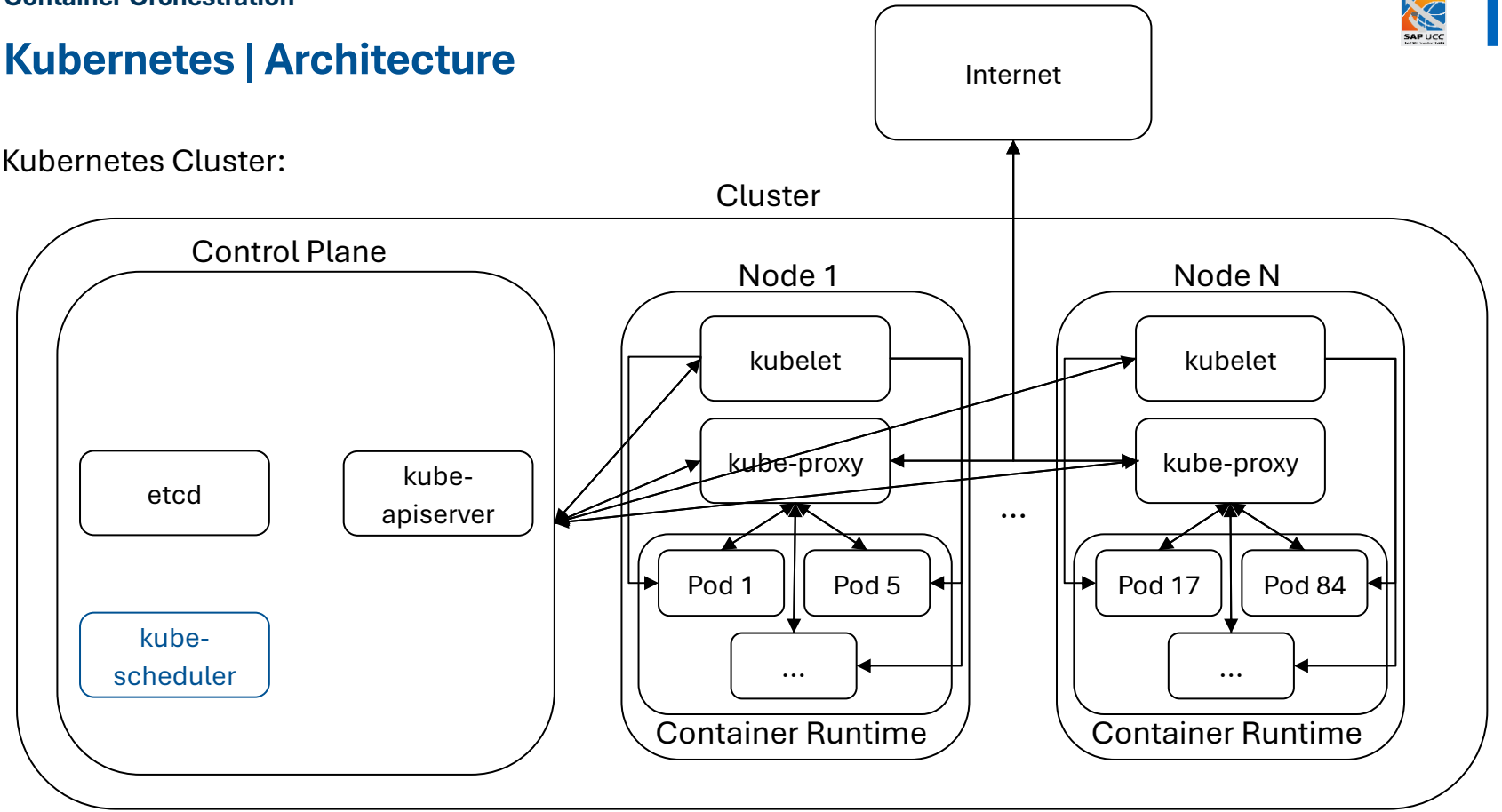
Kubernetes | Architecture

etcd:

- Distributed key-value store
- Stores **all cluster data**
- Made for high availability and consistency

Kubernetes | Architecture

Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

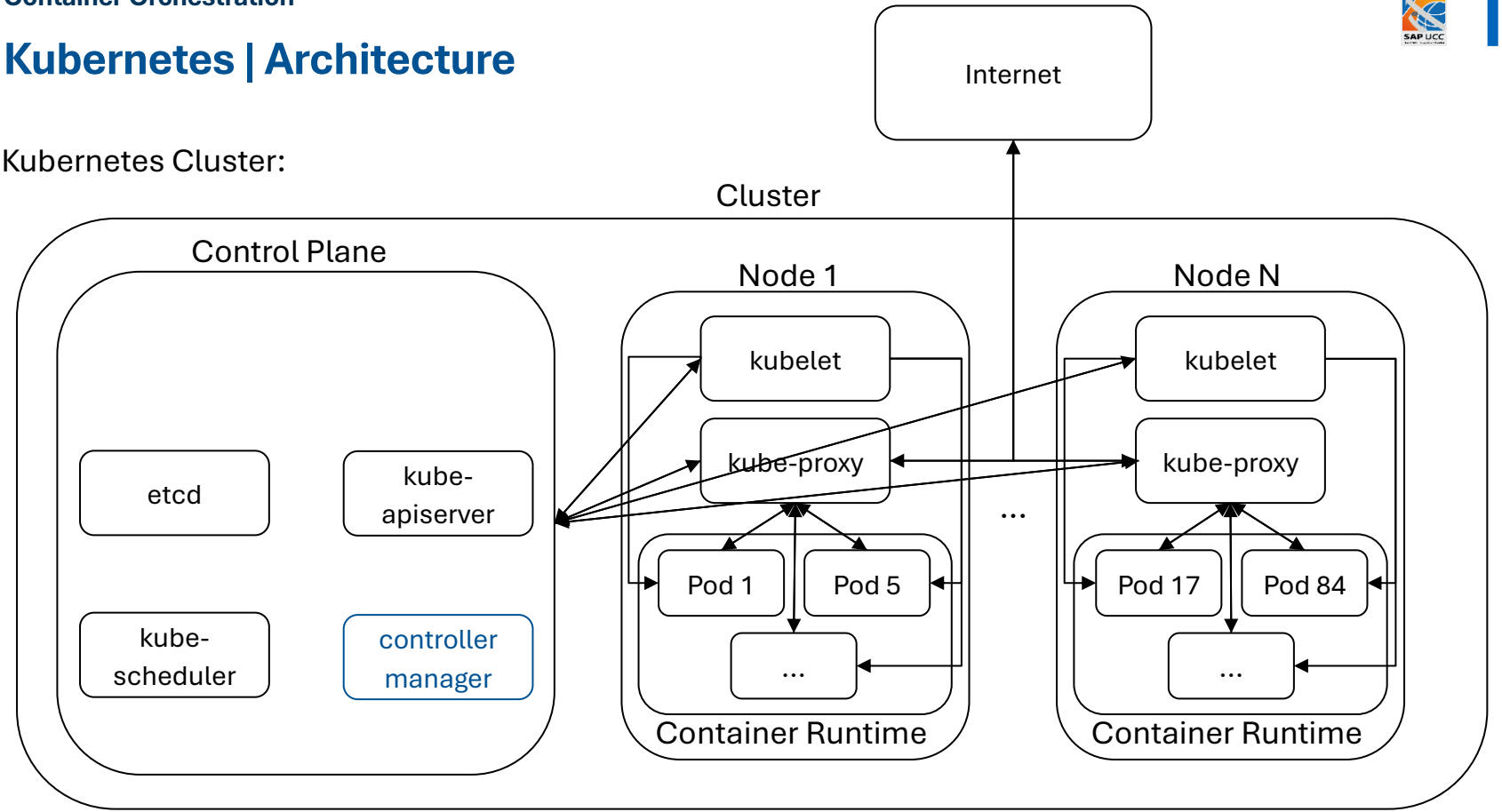
Kubernetes | Architecture

kube-scheduler:

- Watches for new Pods without designated node → **assigns node to run on**
- Node selection factors:
  - Resource requirements
  - Hardware and software policies
  - Inter workload interference

Kubernetes | Architecture

Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

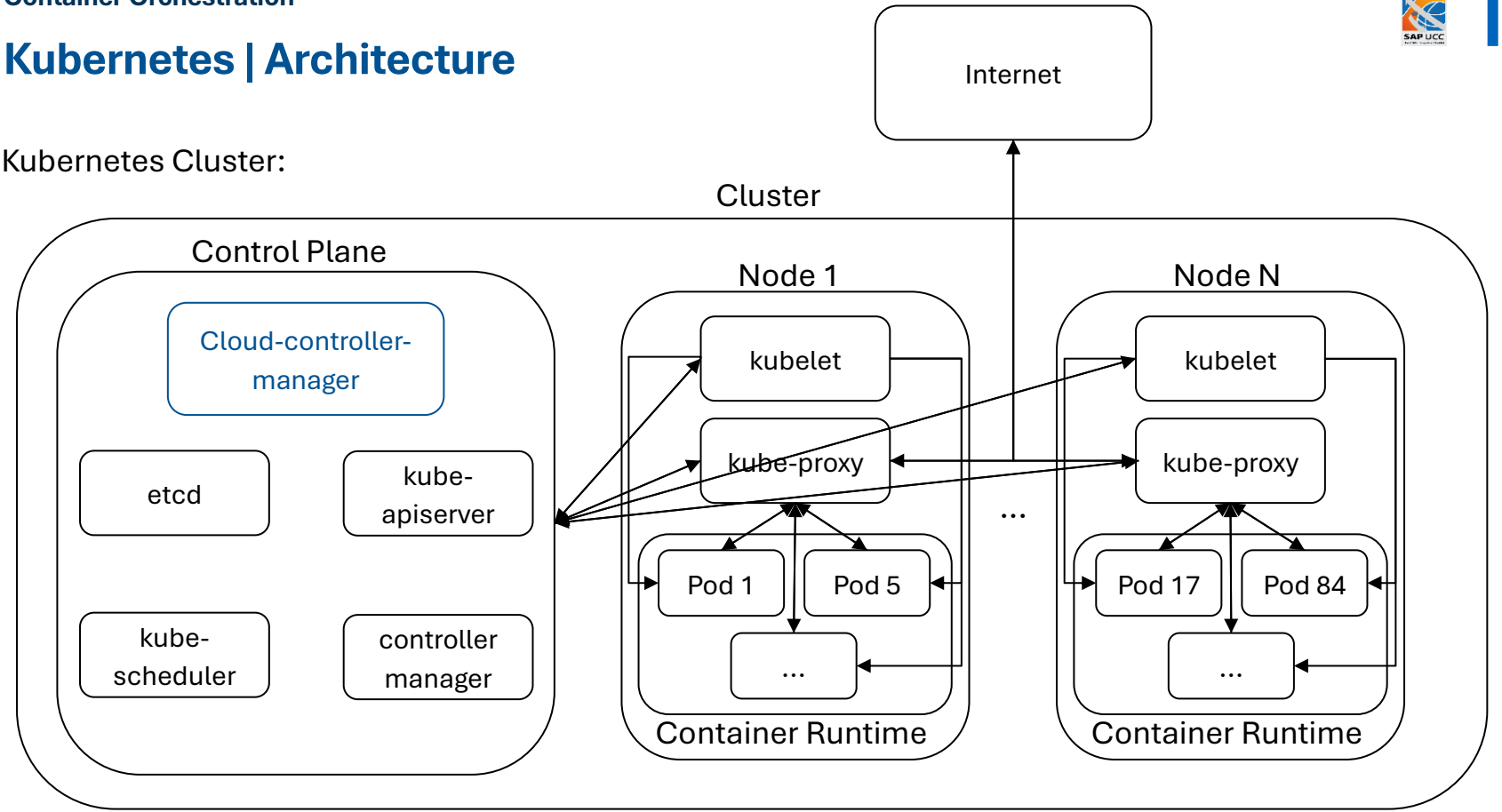
Kubernetes | Architecture

(kube-)controller-manager:

- Runs controller processes
- Controllers are compiled into one binary but are logically separate
- Example controllers:
  - Node controller: monitors if nodes are up and responds in case of downtime
  - Job controller: watches for Job objects (one-off tasks) → creates Pods to run those tasks
  - …

Kubernetes | Architecture

Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

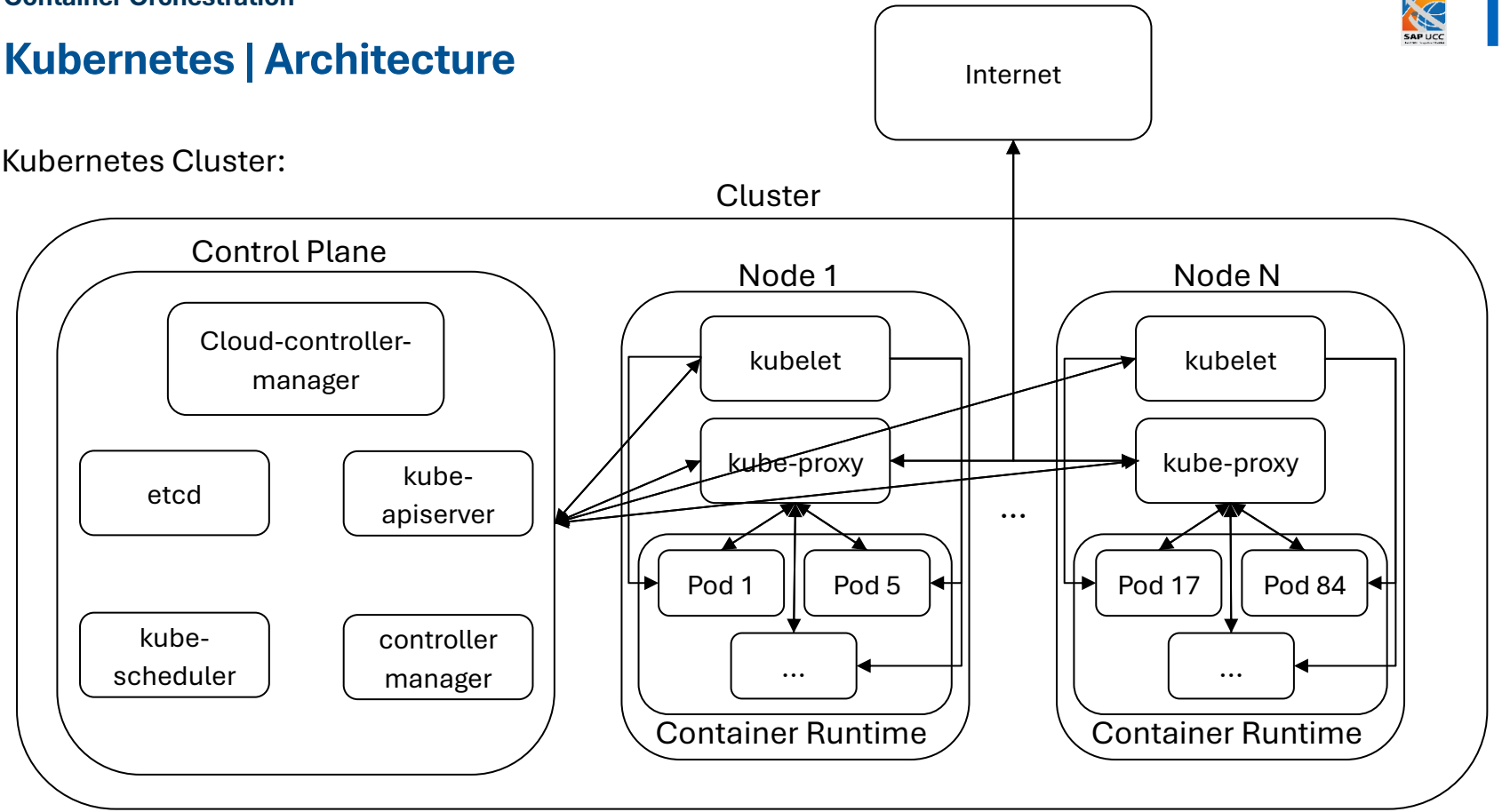
Kubernetes | Architecture

cloud-controller-manager:

- Allows linking a cluster into a cloud provider‘s API
- **Separates** components that interact with the **cloud** from objects that interact with the **local cluster**
- Runs cloud provider specific controllers, e.g.:
  - Route controller: sets up routes in cloud infrastructure
  - Service controller: creates, updates and deletes cloud provider load balancers
  - …

Kubernetes | Architecture

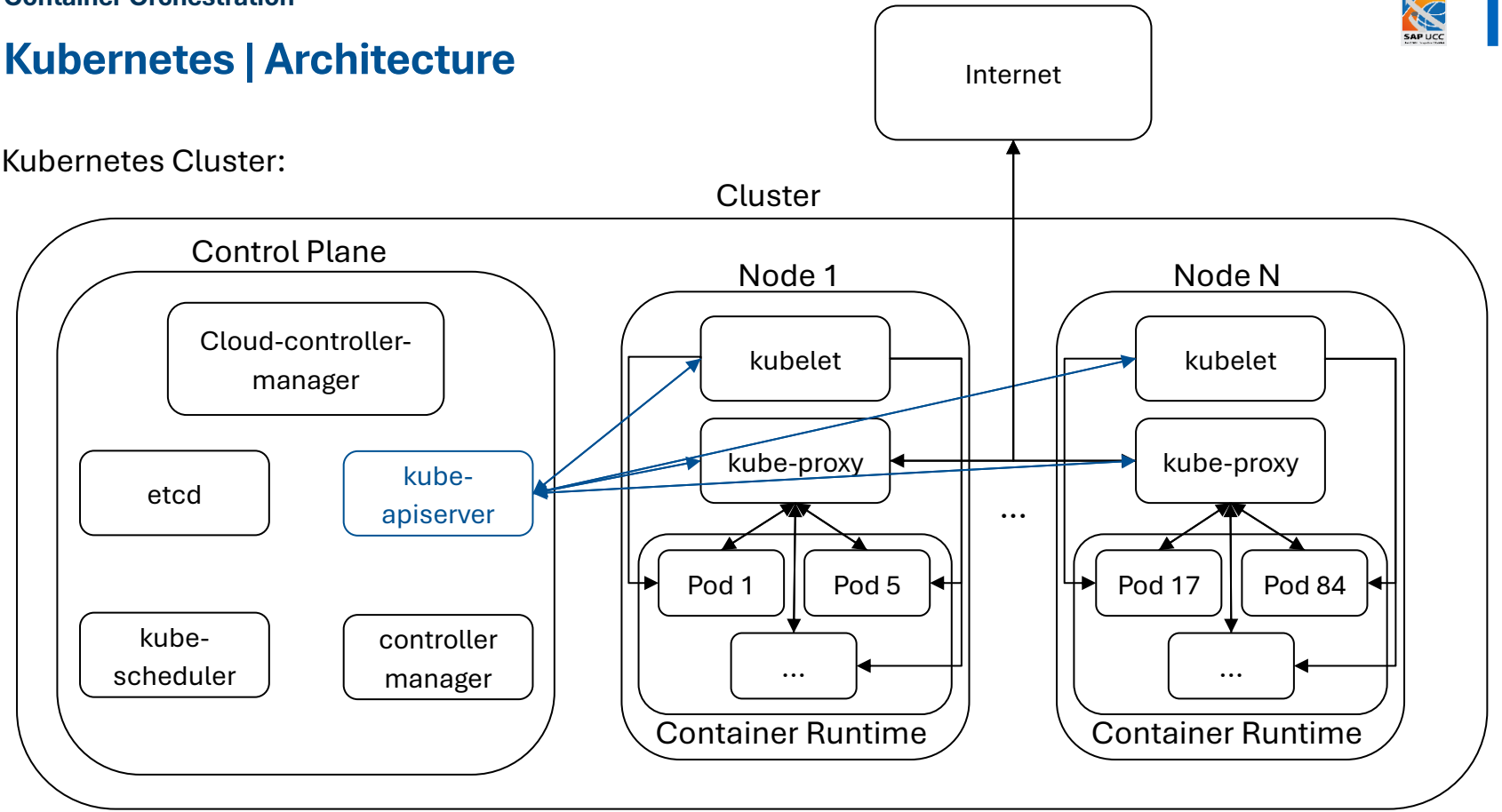
Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

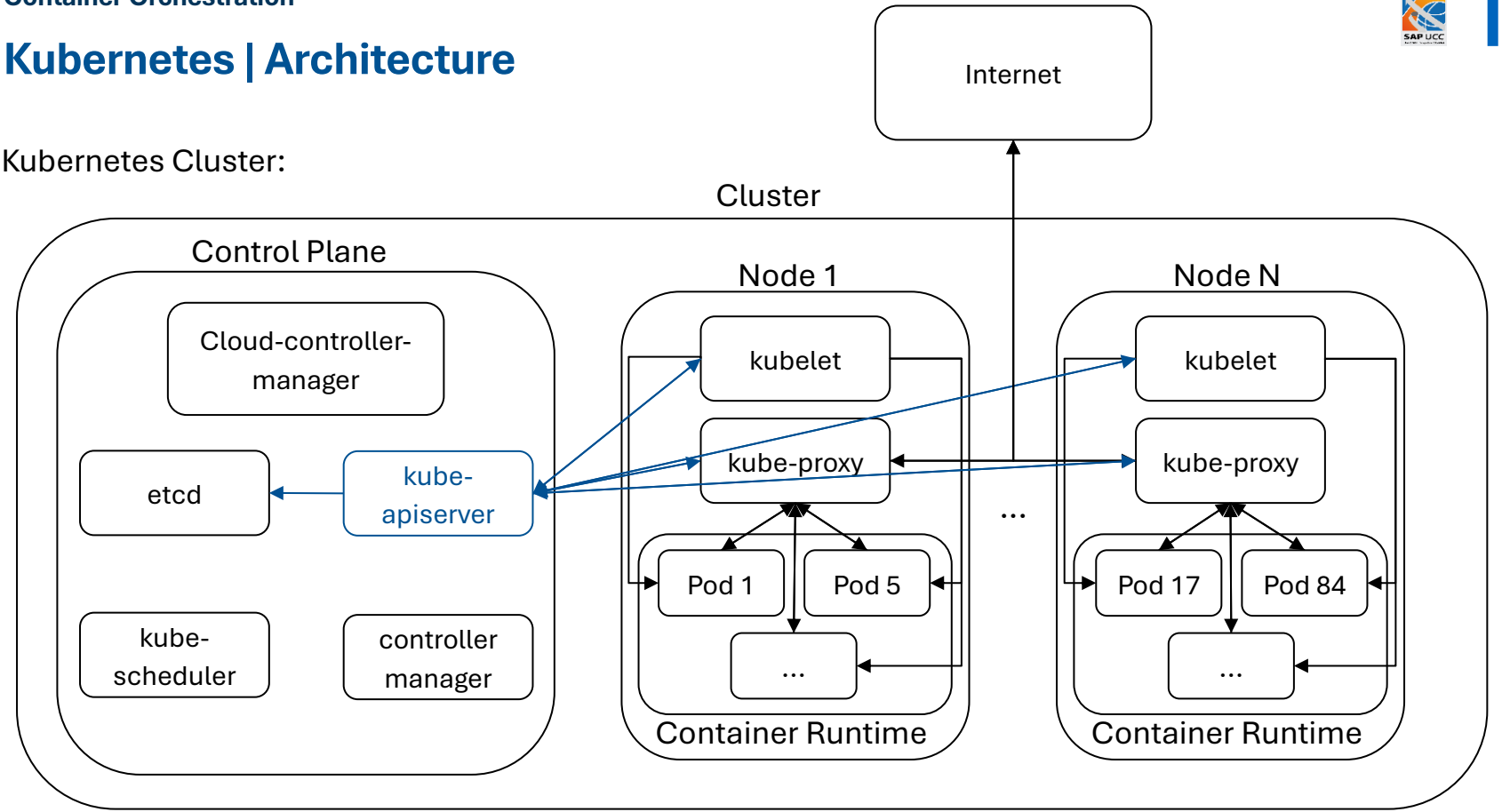
Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

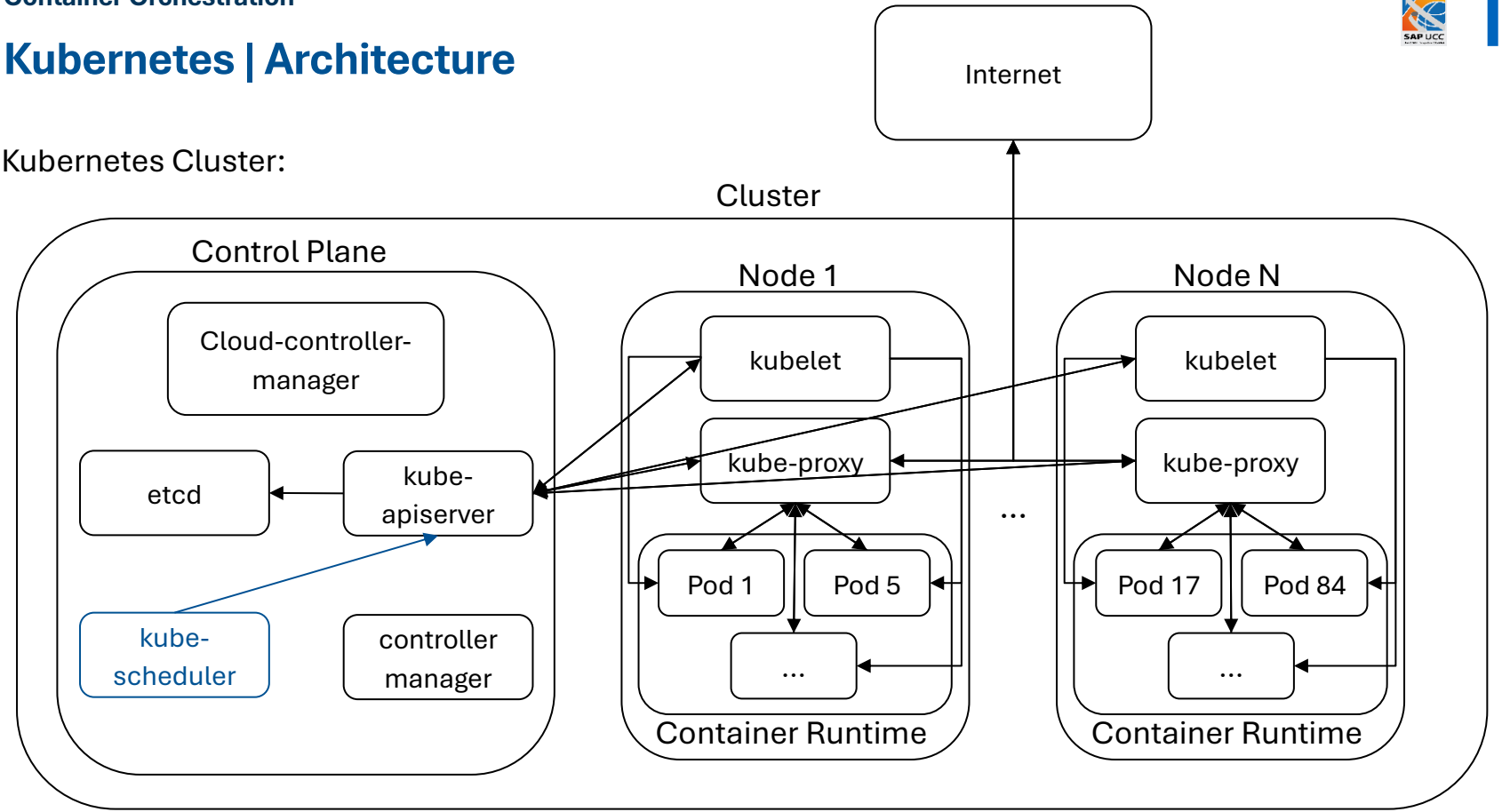
Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

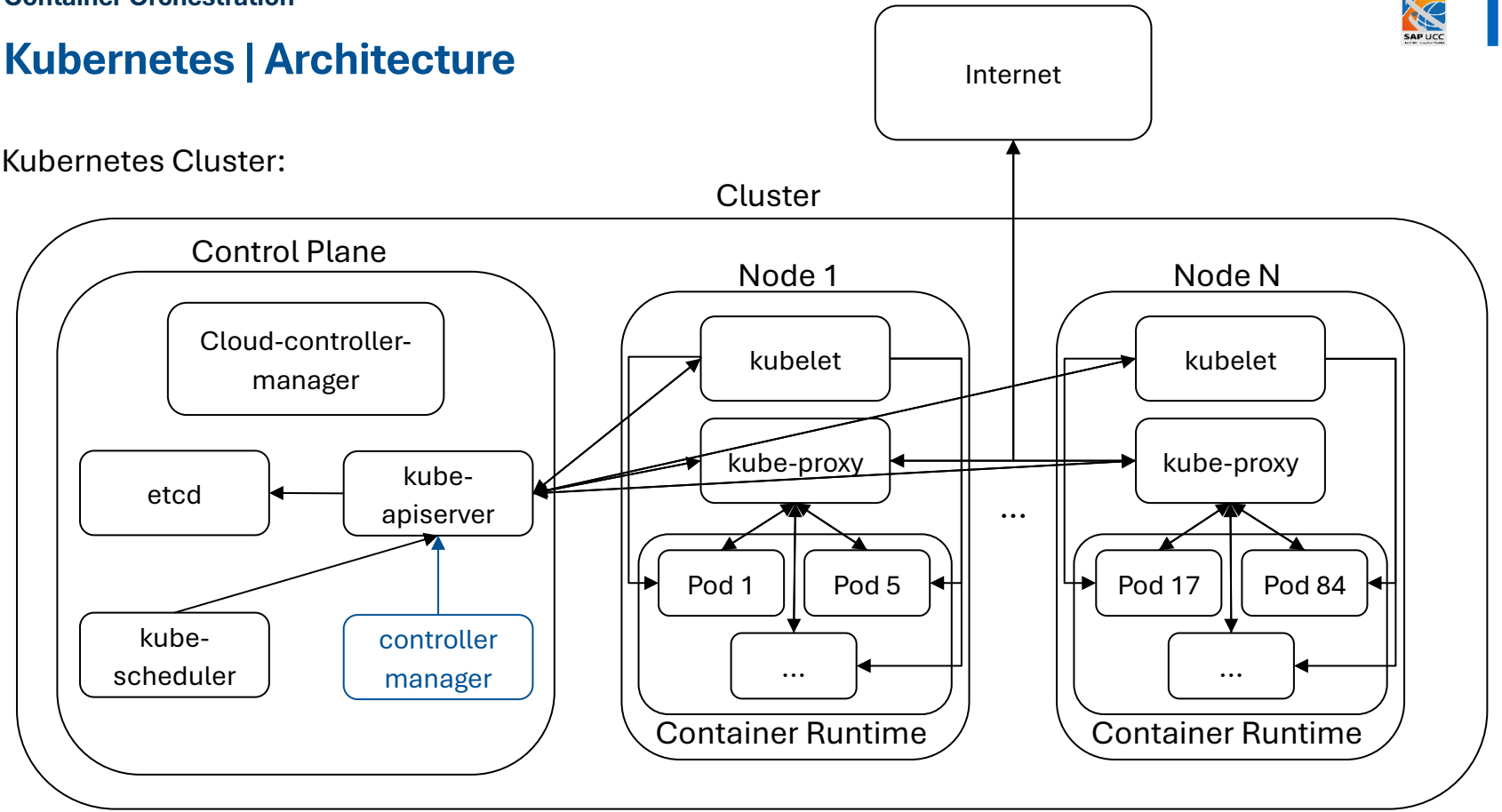
Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

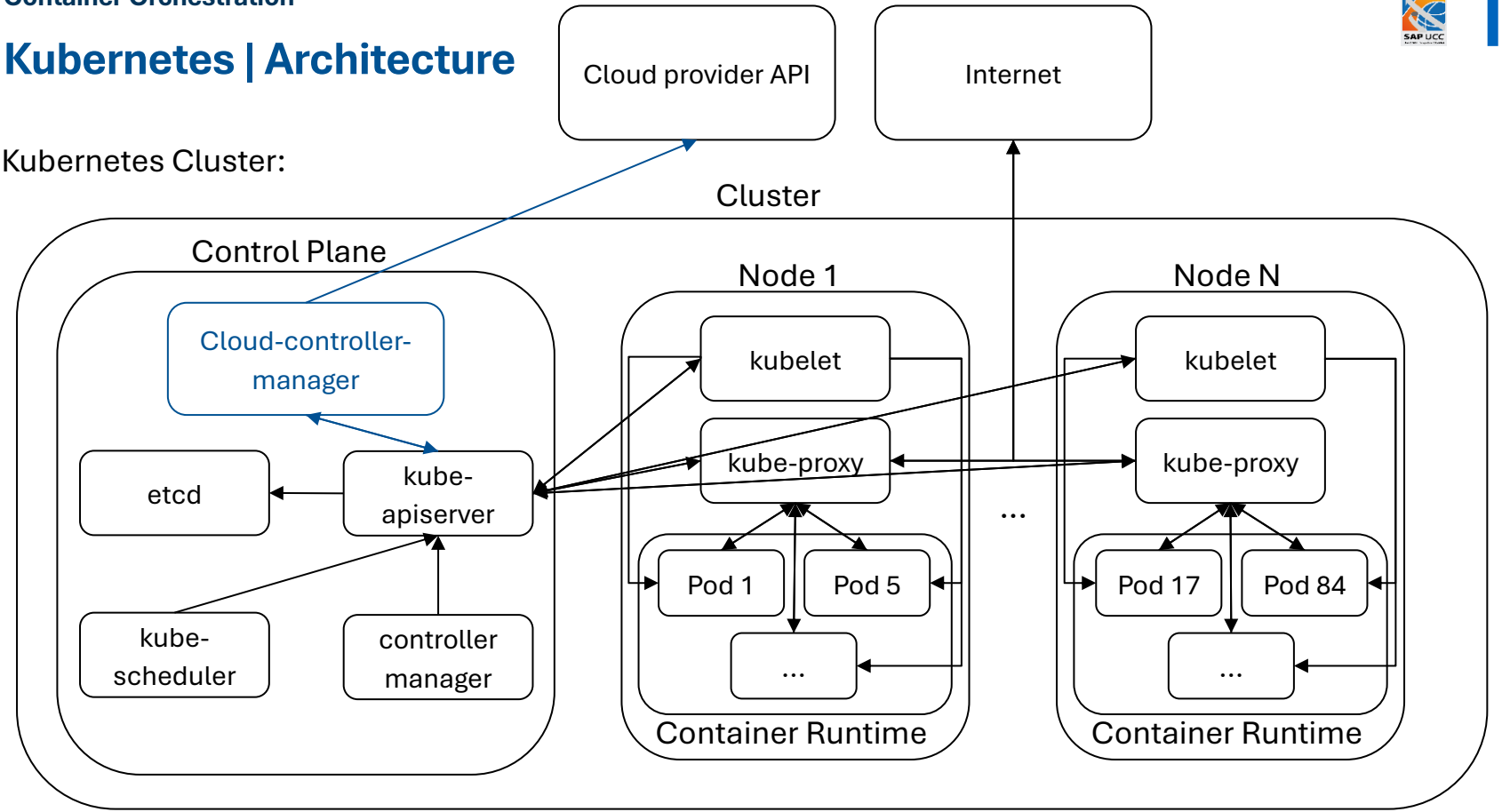
Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

Summary:

- Control plane components:
  - kube-apiserver → exposes Kubernetes API
  - etcd → stores cluster data
  - kube-scheduler → assigns Pods to nodes
  - kube-controller-manager → runs controller processes
  - cloud-controller-manager → runs controller processes related to cloud providers

Kubernetes | Architecture

Summary:

- Node components:
  - kubelet → communicates with Kubernetes-API and ensures given Pods are running
  - kube-proxy → facilitates network communication with pods
  - Container runtime → executes containers
  - Pods →smallest deployable unit, encapsulate containers

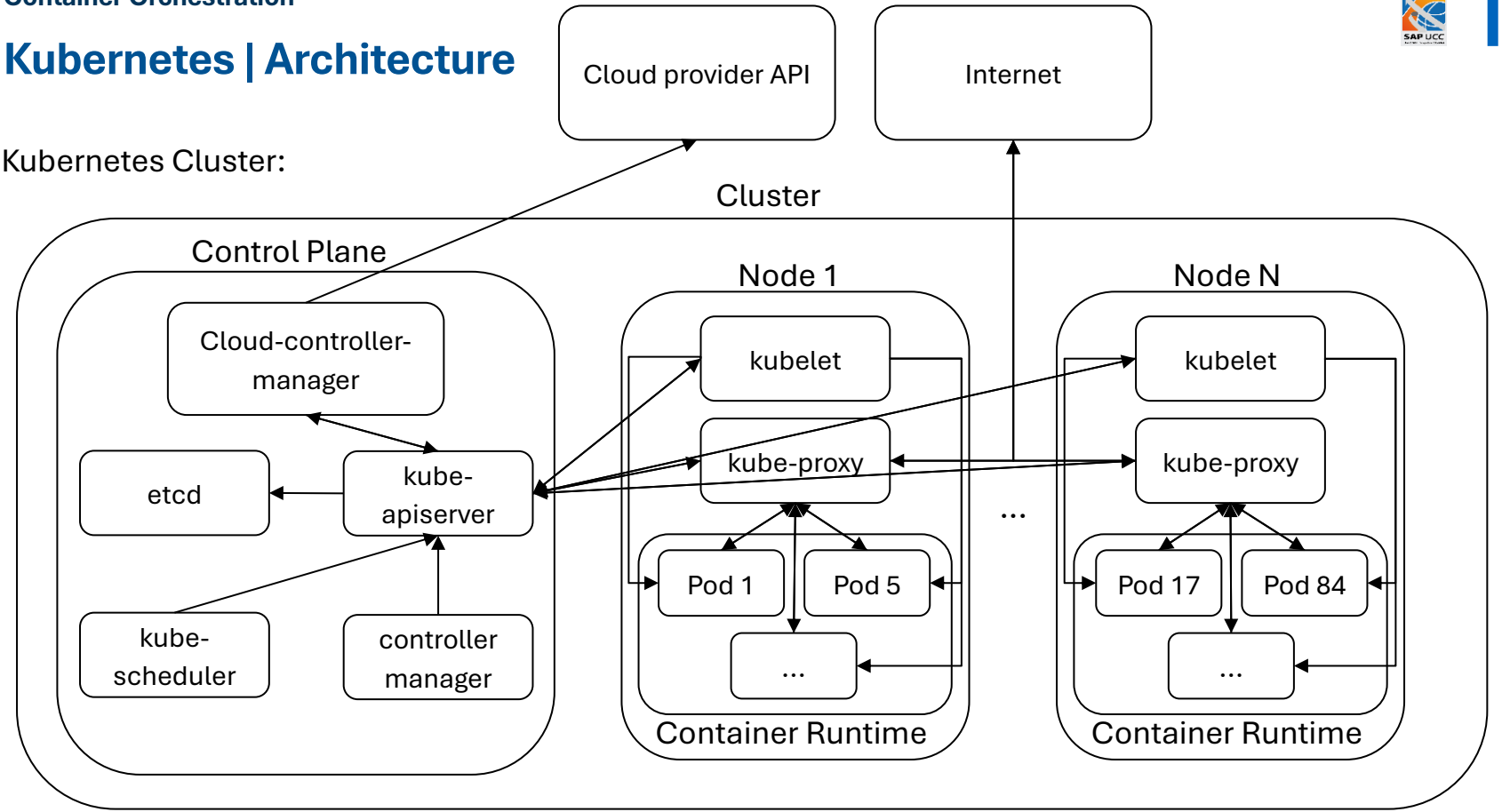
Kubernetes | Architecture

Cluster components:

- Can be deployed in containers (unless it runs containers, i.e., kubelet)
- Kubernetes can manage components running in containers itself
- Clusters can be extended by addons (e.g., DNS, Dashboard, ...)

Kubernetes | Architecture

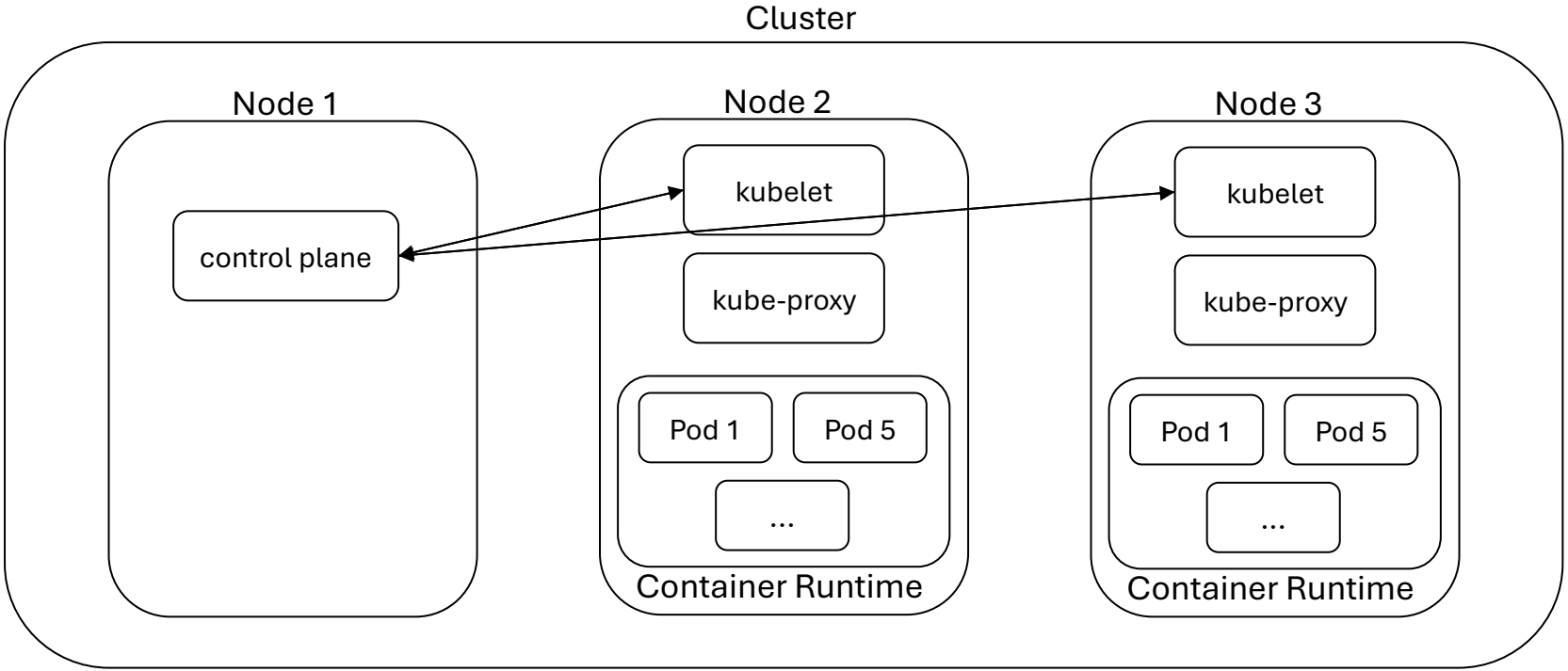
Kubernetes Cluster:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

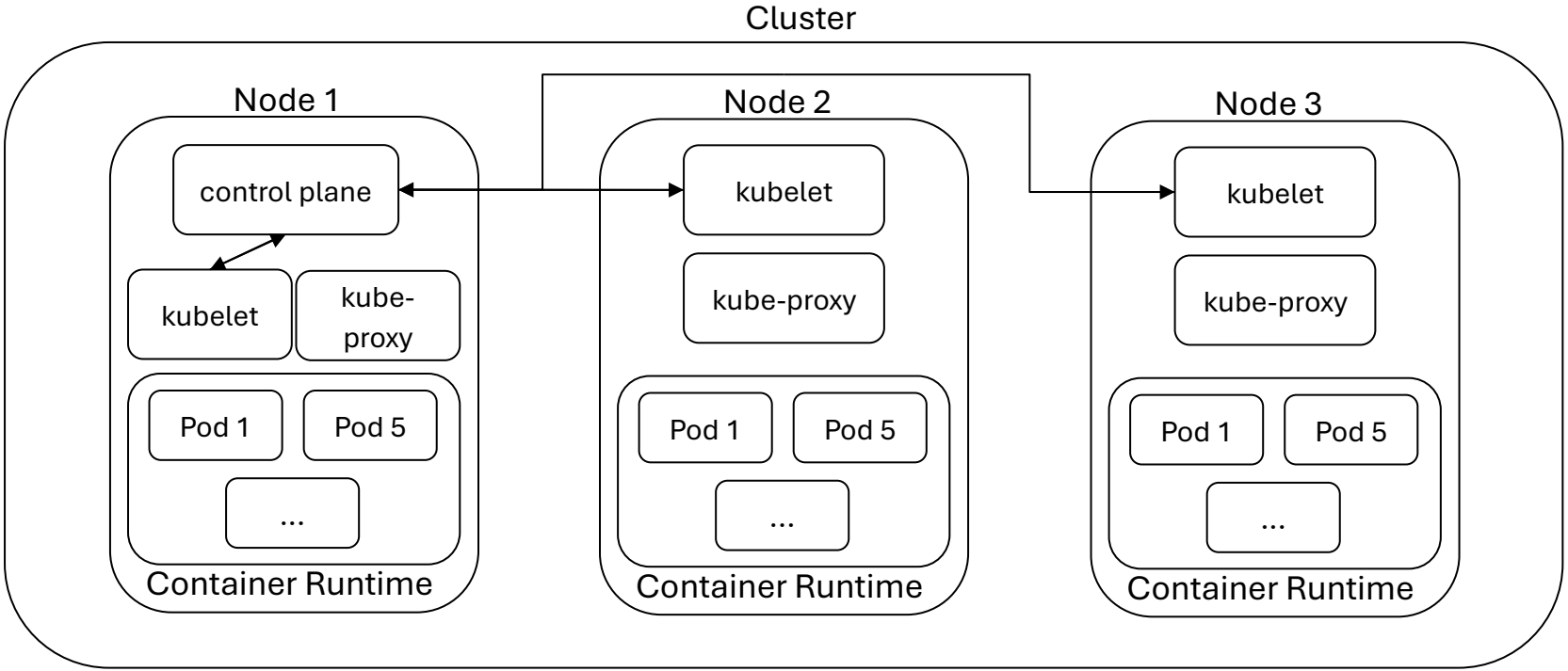
Example Setups:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

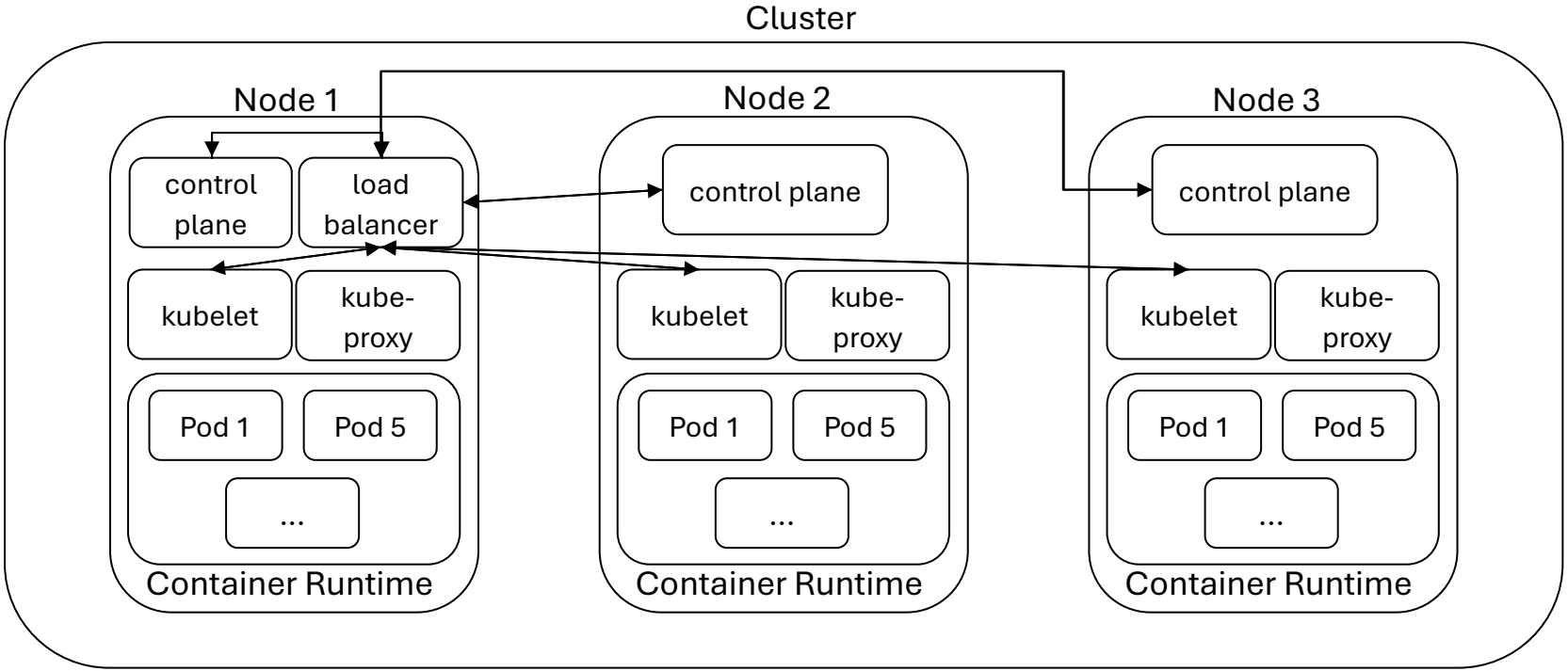
Example Setups:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

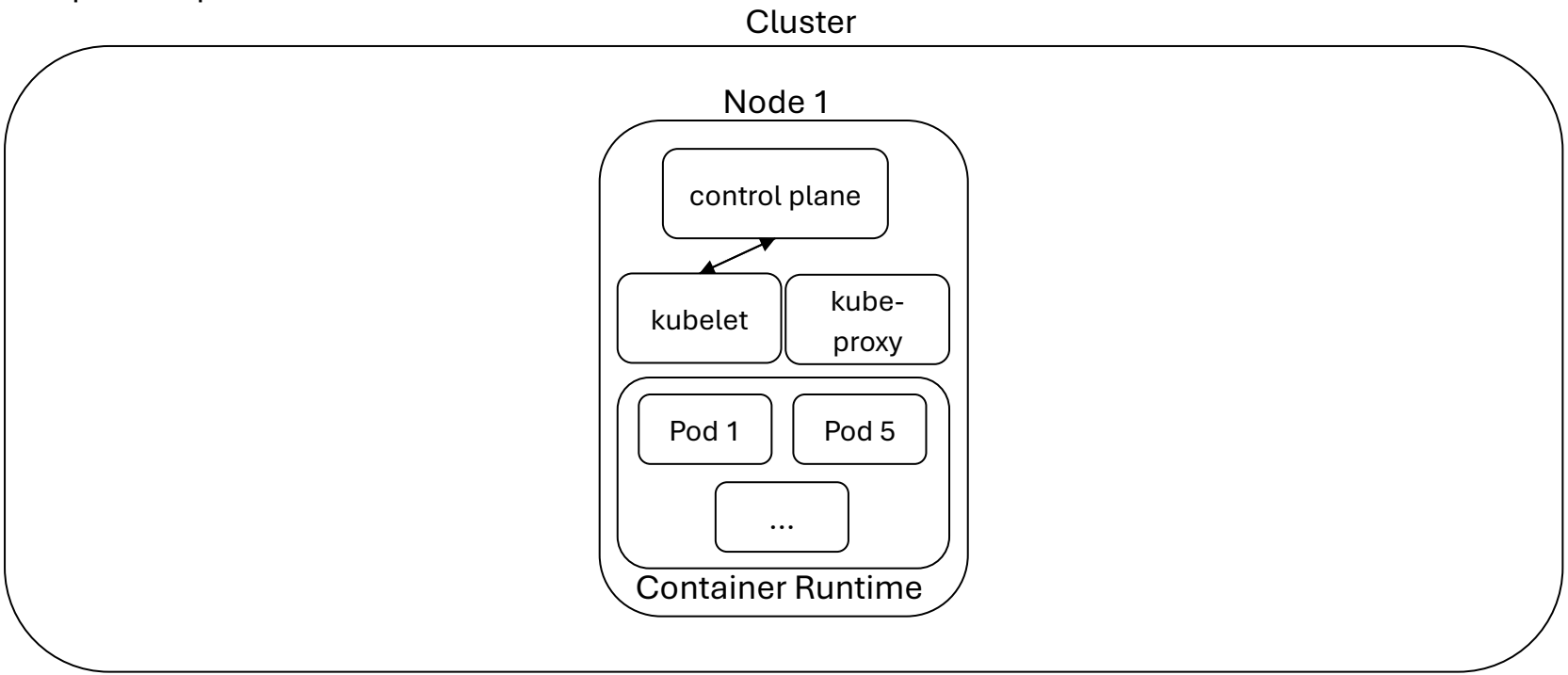
Example Setups:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

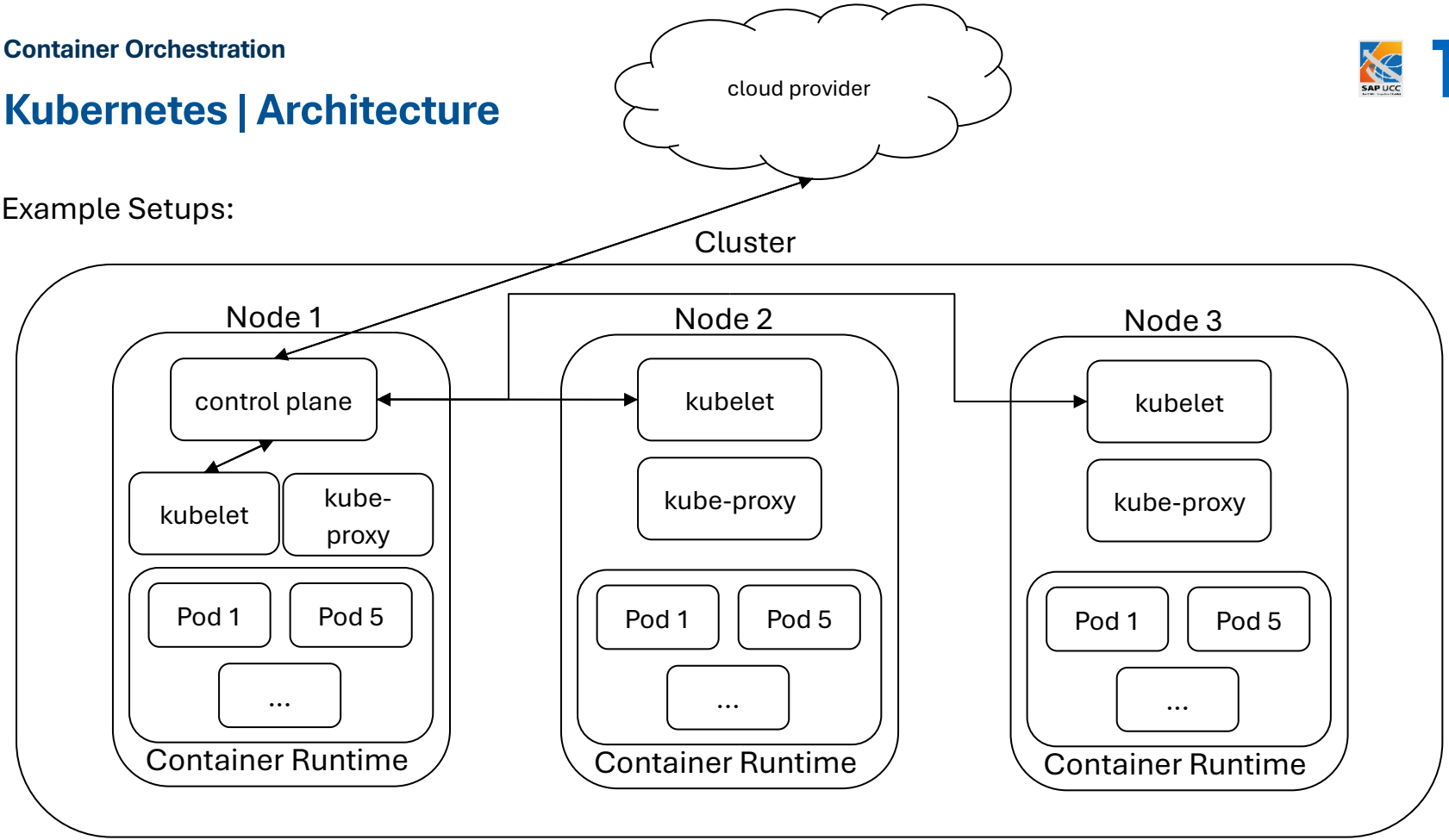
Example Setups:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

Example Setups:



Based on: https://kubernetes.io/docs/concepts/architecture/

Kubernetes | Architecture

Cluster Capabilities:

- Kubernetes is designed to manage up to:
  - 5000 nodes
  - 110 Pods per node
  - 150,000 total Pods
  - 300,000 total containers

Kubernetes | Architecture

Kubernetes Partners:

IBM Cloud

Sources: see references

Kubernetes | Features

We have learned:

- What Kubernetes is
- Kubernetes main components
- Who uses Kubernetes

What we have not learned yet:

Kubernetes | Features

We have learned:

- What Kubernetes is
- Kubernetes main components
- Who uses Kubernetes

What we have not learned yet:

- How to use Kubernetes

Kubernetes | Features

minikube:

- Tool for setting up a **local** (single-node) Kubernetes cluster
- User-friendly (single command setup)
- Supports the latest Kubernetes version
- Open source (https://github.com/kubernetes/minikube/)
- Deploy cluster on VM, container, or bare metal

Sources: https://github.com/kubernetes/minikube/blob/master/images/logo/logo.svg

Kubernetes | Features

minikube:

We can start a local Kubernetes cluster using:

- “minikube start”

Sources: https://github.com/kubernetes/minikube/blob/master/images/logo/logo.svg

Kubernetes | Features

minikube:

We can start a local Kubernetes cluster using:

- “minikube start”

We can receive information about the cluster using:

- “minikube status”

Sources: https://github.com/kubernetes/minikube/blob/master/images/logo/logo.svg

Kubernetes | Features

kubeadm:

- Tool provided by Kubernetes
- Provides functionality to set up clusters
- Follows “best practice”
- Also intended as building block for higher level tools

Sources: https://github.com/kubernetes/website/blob/main/static/images/kubeadm-stacked-color.png

Kubernetes | Features

Kubernetes:

We can interact with the cluster using:

- “kubectl”

Command line interface:

- Enables communication with the control plane
- Utilizes the Kubernetes API

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Kubernetes:

We can see running Pods using:

- “kubectl get pods”
- “kubectl get pods -A” (Pods in all namespaces)

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Kubernetes Namespaces:

- Mechanism to isolate groups of resources in a cluster
- Names are required to be unique in a namespace
- Intended for multi-user or multi-team environments
- Cannot be nested
- Resources can only be in one namespace

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Kubernetes:

We can create Pods using:

- “kubectl run <*name*> --image=<*path-to-image*>”

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Kubernetes:

We can see create Pods using:

- “kubectl run <*name*> --image=<*path-to-image*>”

Alternatively, resources may be created using:

- “kubectl apply –f <*path-to-config-file*>“
- Creates objects from given configuration file (.yaml)

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Kubernetes:

Pod configuration file:

```
apiVersion: v1
kind: Pod
metadata:
  name: example-pod-2
spec:
  containers:
  - name: example-pod-2
    image: nginx:latest
    ports:
    - containerPort: 80
```

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Kubernetes:

We can find out information about objects (e.g., Pods) using:

- “kubectl describe <*resource-name*> <*object-name*>”

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Kubernetes:

To monitor and maintain our cluster, we can:

Retrieve logs of a Pod using:

- “kubectl logs <*object-name*>”

Execute commands in Pods using:

- “kubectl exec <*object-name*> -- <*command*>”

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Kubernetes:

To monitor and maintain our cluster, we can:

Forward system ports to a Pod using:

- “kubectl port-forward <*object-name*> <*port:port*>”

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Kubernetes:

We can remove resources using:

- “kubectl delete <*resource-name*> <*object-name*>”
- “kubectl delete –f <*path-to-config-file*>“

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Labels and Annotations:

Objects can be labeled and annotated:

- Labels:
  - Used to identify and group objects
  - Can be used as selector when listing objects (--selector="...")
- Annotations: key-value pairs, store information

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Labels and Annotations:

Labeled Pod configuration file:

```
apiVersion: v1
kind: Pod
metadata:
  name: example-pod-2
  labels:
    env: "prod"
spec:
  containers:
  - name: example-pod-2
    image: nginx:latest
    ports:
    - containerPort: 80
```

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Resource constraints:

Kubernetes can set bounds on the resources assigned to Pods:

- Requests: lower bound on resources
- Limit: upper bound on resources

Kubernetes can restrict *memory* and *cpu* usage

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Resource constraints:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: example-pod-2-constrained
  labels:
    env: "prod"
spec:
  containers:
  - name: example-pod-2-constrained
    image: nginx:latest
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
    ports:
    - containerPort: 80
```

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

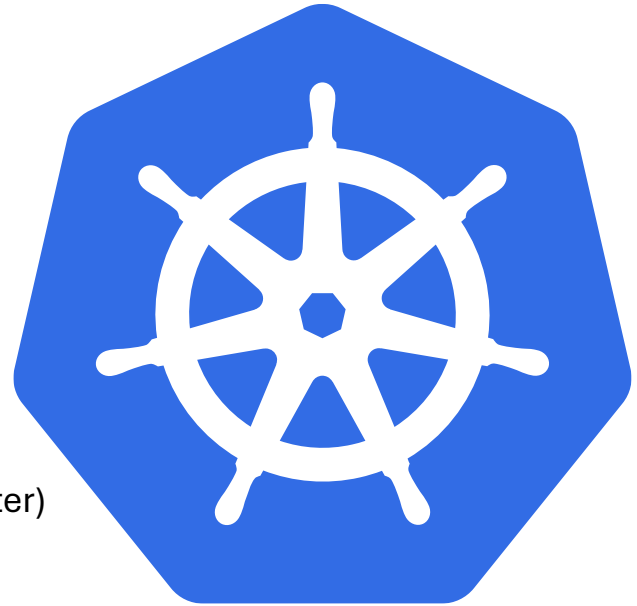
Kubernetes | Features

Services & Service Discovery:

- Kubernetes is very dynamic → IP addresses of Pods may change
- Pods may be running on multiple nodes
- Requires decoupling of network addresses for Pods from nodes

→ Services expose network applications (inside and outside the cluster)

Cluster-aware DNS servers create DNS records for new services



Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Services & Service Discovery:

We can create services using:

- “kubectl expose <*resource-name*> <*object-name*>”
- “kubectl apply –f <*path-to-config-file*>“

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

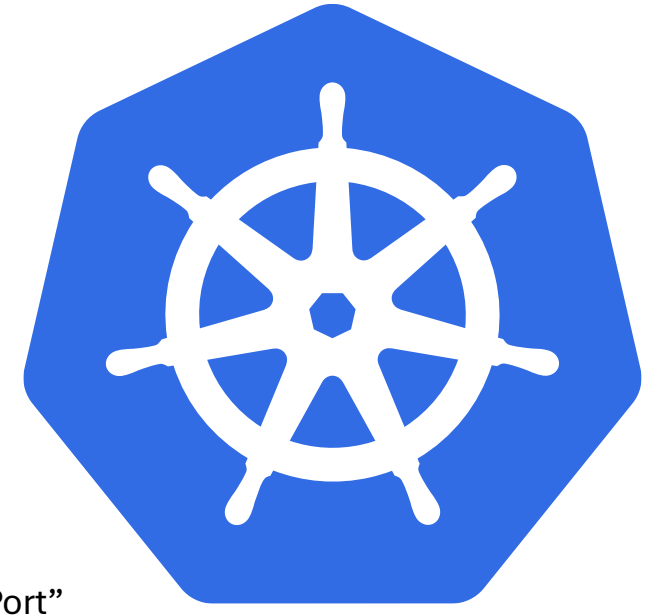
Services & Service Discovery:

We can create services using:

- “kubectl expose <*resource-name*> <*object-name*>”
- “kubectl apply –f <*path-to-config-file*>“

To expose the service outside the cluster:

- “kubectl expose <*resource-name*> <*object-name*> --type=NodePort”



Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

ReplicaSets:

Some applications require multiple concurrent instances, e.g., for:

- Redundancy, Scalability, or Sharding

ReplicaSets manage a set of replicas of a single Pod:

- Ensure a given number of instances are running
- Can be configured to scale automatically

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

ReplicaSets:

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: example-replica
  labels:
    env: prod
spec:
  replicas: 3
  selector:
    matchLabels:
      env: prod
  template:
    metadata:
      labels:
        env: prod
    spec:
      containers:
      - name: example-pod-3
        image: nginx:latest
```

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

ReplicaSets:

We can scale ReplicaSets using:

- “kubectl scale replicasets <*object-name*> --replicas=<*number*>”
- “kubectl apply –f <*path-to-config-file*>“

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

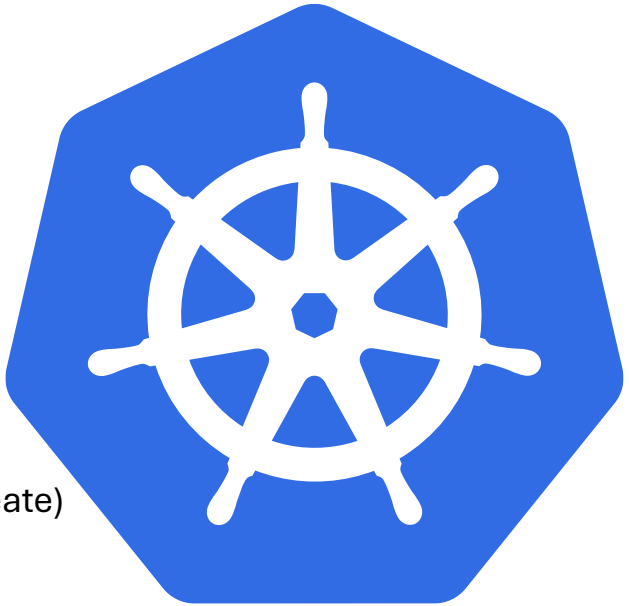
Deployments:

Pods and ReplicaSets are tied to a given container image:

- Need to be fully removed and re-created for updates

Deployments:

- Facilitate rollout process for new updates (Rolling update or Recreate)
- Manage ReplicaSets



Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Deployments :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: example-deployment
  labels:
    env: prod
spec:
  replicas: 3
  selector:
    matchLabels:
      env: prod
  template:
    metadata:
      labels:
        env: prod
    spec:
      containers:
      - name: example-pod-4
        image: nginx:latest
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
```

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Deployments :

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: example-deployment
  labels:
    env: prod
spec:
  replicas: 3
  selector:
    matchLabels:
      env: prod
  template:
    metadata:
      annotations:
        kubernetes.io/change-cause: "Some change"
      labels:
        env: prod
    spec:
      containers:
      - name: example-pod-4
        image: nginx:1.28
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
```

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Deployment:

We can manage the rollout of updates for Deployments using:

- “kubectl rollout status deployments <*object-name*>”
- “kubectl rollout pause deployments <*object-name*>”
- “kubectl rollout resume deployments <*object-name*>”
- “kubectl rollout history deployments <*object-name*>”

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

DaemonSets:

Deployments and ReplicaSets:

- (Generally) agnostic about nodes they run on
- Not as useful for running cluster internal agents or daemons

DaemonSets:

- Run a Pod on each (or a subset) of the nodes in the cluster

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

DaemonSets :

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: example-daemonset
  labels:
    env: prod
spec:
  selector:
    matchLabels:
      env: prod
  template:
    metadata:
      labels:
        env: prod
    spec:
      containers:
      - name: example-pod-5
        image: nginx:latest
```

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Jobs:

Deployments, ReplicaSets, and Daemonsets:

- Meant for active services and applications
- Restart containers on failure

Jobs → Represent Pods meant for one-off tasks

- Execute a Pod → do not restart

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes | Features

Jobs :

```
apiVersion: batch/v1
kind: Job
metadata:
  name: example-job
spec:
  template:
    spec:
      containers:
      - name: example-job
        image: python:latest
        command:
        - python3
        - -c
        - |
          import os, sys
          print("Hello world")
      restartPolicy: Never
```

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Kubernetes

Summary:

- Hyperscalers require a very large number of containers (e.g., to run cloud services)
- Kubernetes can **deploy**, **scale**, and **manage** containerized applications
- Components:
  - Control plane: kube-apiserver, etcd, kube-scheduler, kube-controller-manager, cloud-controller-manager
  - Nodes: kubelet, kube-proxy, Container runtime, Pods

Kubernetes

Summary:

- Important Kubernetes objects:
  - Pods
  - Services
  - ReplicaSets
  - Deployments
  - DaemonSets
  - Jobs

Source: https://github.com/kubernetes/kubernetes/blob/master/logo/logo.svg

Sources

- https://docs.docker.com/get-started/docker-overview/
- https://github.com/opencontainers/image-spec/blob/main/spec.md
- https://opencontainers.org/
- https://docs.docker.com/reference/dockerfile
- https://www.man7.org/linux/man-pages/man7/namespaces.7.html
- https://linux.die.net/man/1/unshare
- https://www.man7.org/linux/man-pages/man7/cgroups.7.html
- https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-registry/
- https://github.com/opencontainers/distribution-spec/blob/main/spec.md
- https://docs.docker.com/build/concepts/dockerfile/
- https://github.com/opencontainers/runtime-spec/blob/main/spec.md
- https://docs.docker.com/engine/daemon/alternative-runtimes/

Sources

- https://github.com/containers/podman
- https://podman.io/
- Hightower, K., Burns, B., Beda, J., & Demmig, T. (2018). *Kubernetes: Eine kompakte Einführung* (1. Auflage.). dpunkt.verlag.
- https://www.redhat.com/de/topics/cloud-computing/what-is-a-hyperscaler
- https://www.ibm.com/de-de/think/topics/hyperscale
- https://cloud.google.com/learn/what-is-a-cloud-service-provider?hl=en
- https://www.redhat.com/en/topics/cloud-computing/what-are-cloud-providers
- https://www.flaticon.com/free-icon/container_7115168?term=container&page=1&position=1&origin=tag&related_id=7115168
- https://github.com/orgs/networkx/repositories

Sources

- https://www.ibm.com/think/topics/kubernetes-history
- https://kubernetes.io/docs/setup/best-practices/cluster-large/
- https://queue.acm.org/detail.cfm?id=2898444
- https://www.cncf.io/reports/kubernetes-project-journey-report/
- https://kubernetes.io/docs/concepts/architecture/
- https://etcd.io/
- https://kubernetes.io/docs/concepts/workloads/pods/
- https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet/
- Logos: https://commons.wikimedia.org/wiki/File:Google_Cloud_logo.svg, https://commons.wikimedia.org/wiki/File:IBM_Cloud_logo.png, https://commons.wikimedia.org/wiki/File:Amazon_Web_Services_Logo.svg, https://commons.wikimedia.org/wiki/File:AlibabaCloudLogo.svg, https://commons.wikimedia.org/wiki/File:Microsoft_Azure.svg

Sources

- https://kubernetes.io/docs/setup/best-practices/cluster-large/
- https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/ha-topology/
- https://github.com/kubernetes/minikube/
- https://minikube.sigs.k8s.io/docs/
- https://kubernetes.io/docs/tasks/tools/#kubectl
- https://github.com/kubernetes/kubeadm
- https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/